Project Report

Deploy Two Web Application on the Same Linux Server

Performed by:

Huỳnh Ân

Student information:

24560064

24560064@gm.uit.edu.vn

Supervised by:

Trần Thị Dung

Cource Name:

Operation System

Time:

24/8/2025

TABLE OF CONTENTS

Abberviation 3

Table of Figure 4

I. Introduction 6

1. Definition 6

2. Components 6

3. Operation 7

Case 1: Serving Static Content 7

Case 2: Serving Dynamic Content via MariaDB Database 10

II. Implementation 16

1. Topology 16

Basic Setup Topology: 16

ADVANCE Setup Topology: 16

2. Installation 17

3. Configuration 25

1. Basic Web Deployment 25

2. Advance web deployment with database 38

III. Results 50

1. Basic web deployment 50

2. Advance web deployment with database 51

IV. References 54

ABBERVIATION

- **DHCP**: Dynamic Host Configuration Protocol
- **DNS**: Domain Name System
- **HTTP**: HyperText Transfer Protocol
- **IP**: Internet Protocol
- **NAT**: Network Address Translation
- **SQL**: Structured Query Language
- **UFW**: Uncomplicated Firewall
- **VM**: Virtual Machine

TABLE OF FIGURE

Figure 1: DNS Query Flowchart (Case 1& 2) 8
Figure 2: HTTP Request Flowchart (Case 1 & 2) 9
Figure 3:Static Delivery Flowchar 10
Figure 4: Flrewall Security Flowchar (Case 1) 10
Figure 5: DNS Query Flowchart (Case 1 & 2) 11
Figure 6: HTTP Request Flowchart (Case 1 & 2) 12
Figure 7: Database Query Flowchart 13
Figure 8: Dynamic Content Delivery Flowchart 14
Figure 9: Firewall Security Flowchart(Case 2) 15
Figure 10: Kali Linux System and Package Verification 18
Figure 11: UFW Firewall Configuration for Web Server 19
Figure 12: Apache2 and BIND9 Installation & Service Status 20
Figure 13: Apache2 and BIND9 Installation & Service Status 21
Figure 14: PHP Configuration 22
Figure 15: PHP and MariaDB Client Installation 23
Figure 16: MariaDB Installation on Database Server 24
Figure 17: Git Clone DVWA on Database Server 25
Figure 18: Default Network and DNS Settings 27
Figure 19: Connectivity Test (Gateway & Public IP) 28
Figure 20: Port Configuration for Listen 100 29
Figure 21: Configuration virtual host for ws1.com on Apache 30
Figure 22: Configuration virtual host for ws2.com on Apache 31
Figure 23: Static Page for Basic Setup 32
Figure 24: Reload and Restart for Static Page 33
Figure 25: BIND9 Define Zone Configuration 34
Figure 26: Zone file Configuration for ws1.com 35
Figure 27: Zone file Configuration for ws2.com 36
Figure 28: Status of BIND9 37
Figure 29: Firewall Configuration for ports allowed on Web and DNS Server 38

Figure 30: Default Database Settings 40
Figure 31: Bind Address Configuration of MaridaDB 41
Figure 32: DVWA Database Status in MariaDB 42
Figure 33: Firewall Configuration for ports allowed on Database Server 43
Figure 34: DVWA Database Configuration for ws1.com 44
Figure 35: DVWA Database Configuration for ws2.com 45
Figure 36:MariaDB Connectivity on Database Server 46
Figure 37: Configuration virtual host for ws1.com by DVWA on Apache 48
Figure 38: Configuration virtual host for ws2.com by DVWA on Apache 49
Figure 39: DNS Resolution for Window Client 50
Figure 40: Nslookup for ws1.com and ws2.com 51
Figure 41: Static Content Result 51
Figure 42: DVWA Setup Page 52
Figure 43: DVWA Login Page 53
Figure 44: DVWA Web Application 53

I. INTRODUCTION

1. DEFINITION

Web application deployment involves configuring server infrastructure to host websites, manage domain name resolution, and, in advanced setups, serve dynamic content by integrating with databases. This project deploys two websites, **`ws1.com`** and **`ws2.com`**, on Kali Linux virtual machines (VMs) using Apache2 as the web server and BIND9 as the DNS server. The basic setup uses a single Kali Linux VM (IP 192.168.35.132) running Apache2 on port 100 to serve static HTML content and BIND9 on port 53 to resolve domain names. The advanced setup adds a second Kali Linux VM (IP 192.168.35.133) running MariaDB on port 3306 to store and provide dynamic data for both websites, accessible via the user **huynhan** with password **huynhan1812**. The project aims to develop essential skills in server administration, web hosting, DNS configuration, and database integration in a Kali Linux environment, ensuring secure service delivery with UFW firewalls. The scope covers installing and configuring Apache2 and BIND9 for static content delivery in the basic setup and integrating a remote MariaDB database for dynamic content in the advanced setup, ensuring accessibility via domain names and secure data retrieval.

2. COMPONENTS

The system consists of several components working together to deliver web content, resolve domain names, and provide dynamic data in the advanced setup. Each component is described below in a single paragraph, focusing on theoretical roles and interactions without implementation details.

Kali Linux Server (Web/DNS Server): This virtual machine serves as the primary platform for hosting web and DNS services, running a Linux-based operating system to manage network traffic and service processes. In the basic setup, it hosts both Apache2 and BIND9, sharing resources like CPU and memory to handle HTTP and DNS requests. In the advanced setup, it focuses on web and DNS tasks, offloading database operations to a separate server. It interacts with clients by processing DNS queries and HTTP requests, forwarding database queries to the database server in the advanced setup.

Apache2 (Web Server): Apache2 hosts **`ws1.com`** and **`ws2.com`** using virtual hosts, serving static content in the basic setup and dynamic content (e.g., DVWA) in the advanced setup via scripting languages like PHP. It processes HTTP requests from clients, matching domains to the correct virtual host, and interacts with BIND9 for domain resolution and the database server for dynamic data, ensuring seamless content delivery.

BIND9 (DNS Server): BIND9 resolves **`ws1.com`** and **`ws2.com`** to IP addresses, enabling clients to locate the web server. Operating alongside Apache2 in the basic setup, it handles UDP-based DNS queries and shares network resources. In the advanced setup, it continues to resolve domains, supporting Apache2's interactions with the database server indirectly through client requests.

Client Machine (Windows): The Windows client tests the system by sending DNS queries to BIND9 and HTTP requests to Apache2 via a browser or command-line tools. In the advanced setup, it accesses dynamic content generated by Apache2's database queries, serving as the interface to verify system functionality.

UFW (Web/DNS Server): UFW secures the web/DNS server by restricting traffic to authorized ports (e.g., DNS, HTTP), using iptables to protect Apache2 and BIND9. In the advanced setup, it also secures the database server, allowing specific traffic between servers, ensuring secure communication across all components.

Kali Linux Server (Database Server, Advanced Setup): This second virtual machine hosts the database software in the advanced setup, managing data storage and retrieval for dynamic web content. It communicates with the web server over a network protocol, providing data for applications like DVWA.

MariaDB (Database Software, Advanced Setup): MariaDB manages databases, storing dynamic data for **`ws1.com`** and **`ws2.com`**, and processing SQL queries from Apache2 via a client connection. It ensures data integrity and security through user authentication, interacting with Apache2 to deliver dynamic content securely.

3. OPERATION

This section outlines the client's interaction with the system for accessing static and dynamic content, with each step listed separately and described in a single cohesive paragraph, following the reference document's clear, step-by-step format.

CASE 1: SERVING STATIC CONTENT

1. **DNS Query**: The Windows client sends a DNS query to the Web/DNS server (192.168.35.132:53) for **`ws1.com`** or **`ws2.com`** using a browser **`or nslookup ws1.com 192.168.35.132`**. BIND9 processes the query using **`/etc/bind/named.conf.local`**, which specifies the zone files for domain resolution:

```
zone "ws1.com" {
    type master;
    file "/etc/bind/zones/db.ws1.com";
};
zone "ws2.com" {
    type master;
    file "/etc/bind/zones/db.ws2.com";
};
```

The zone file **`/etc/bind/zones/db.ws1.com`** (or **`db.ws2.com`**) maps the domain to the IP:

```
@       IN    A     192.168.35.132
www     IN    A     192.168.35.132
```

The **`A`** records direct BIND9 to return **`192.168.35.132`**, enabling the client to send an HTTP request to the Web/DNS server. This configuration ensures that both **`ws1.com`** and **`ws2.com`** resolve to the same IP, relying on Apache2's virtual hosting to differentiate them. The process uses the Linux kernel's UDP stack, and UFW allows port 53 (**`sudo ufw allow 53`**), securing the query. Successful resolution is verified by **`nslookup ws1.com 192.168.35.132`** or **`nslookup ws2.com 192.168.35.132`**. The **`zone`** directive in **`/etc/bind/named.conf.local`** is critical for linking domain names to their zone files, and

the **A** records in the zone files ensure accurate IP resolution, forming the first step in directing client requests to Apache2.

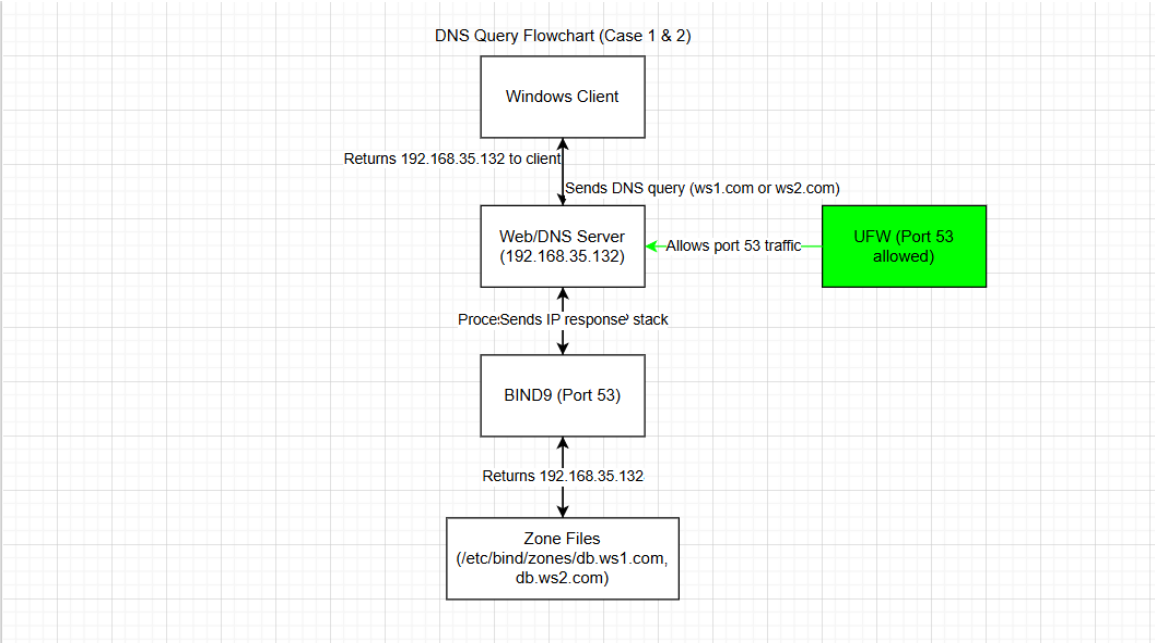

Figure 1: DNS Query Flowchart (Case 1& 2)

2. **HTTP Request**: The client sends an HTTP request to **`http://ws1.com:100`** or **`http://ws2.com:100`** via a browser, including the **`Host`** header (e.g., **`Host: ws1.com`**) in the TCP packet. Apache2, configured to listen on port 100 via **`/etc/apache2/ports.conf`** (**`Listen 100`**), receives the request and uses name-based virtual hosting to identify the domain. The virtual host file **`/etc/apache2/sites-available/ws1.com.conf`** specifies:

```
<VirtualHost *:100>
    ServerName ws1.com
    ServerAlias www.ws1.com
    DocumentRoot /var/www/ws1/public_html
    ErrorLog ${APACHE_LOG_DIR}/ws1-error.log
    CustomLog ${APACHE_LOG_DIR}/ws1-access.log combined
</VirtualHost>
```

Similarly, **`/etc/apache2/sites-available/ws2.com.conf`** specifies **`ServerName ws2.com`** and **`DocumentRoot /var/www/ws2/public_html`**. The **`ServerName`** directive matches the **`Host`** header (**`ws1.com`** or **`ws2.com`**), and since both websites share the same IP (192.168.35.132) and port (100), Apache2 uses **`ServerName`** to route the request to the correct **`DocumentRoot`** (**`/var/www/ws1/public_html`** or **`/var/www/ws2/public_html`**). The **`Listen 100`** command ensures Apache2 accepts requests on port 100, and **`ServerName`** enables differentiation of requests for multiple domains on the same server. UFW allows port 100 (**`sudo ufw allow 100/tcp`**), securing the connection. The TCP stack ensures reliable delivery, and browser access to the URLs verifies the request.

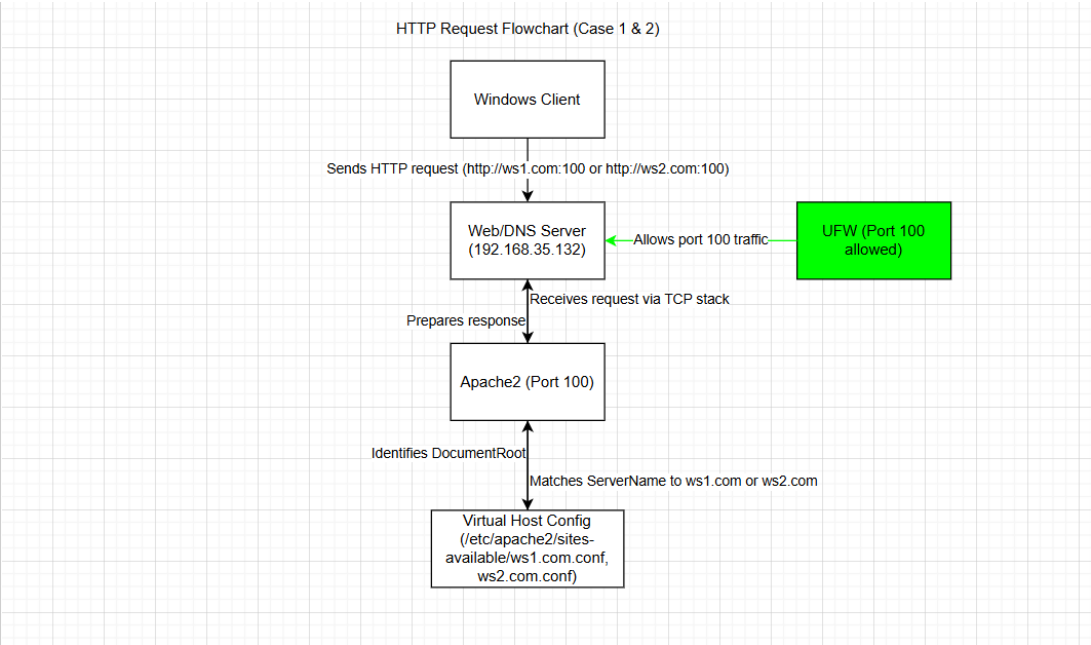

Figure 2: HTTP Request Flowchart (Case 1 & 2)

HTTP_FLowchart.png

3. **Content Delivery**: Apache2 retrieves **`index.html`** from the specified **`DocumentRoot`** (e.g., **`/var/www/ws1/public_html/index.html`**), which contains static content like **`<h1>Welcome to WS1!</h1>`**. The **`DocumentRoot`** directive in **`/etc/apache2/sites-available/ws1.com.conf`** points Apache2 to the correct directory for **`ws1.com`**, ensuring it serves the appropriate file. File permissions are set to 755 with owner **`www-data`** using:

```
sudo chown -R www-data:www-data /var/www
sudo chmod -R 755 /var/www
```

These commands ensure Apache2, running as **`www-data`**, can read the files, with **`chown`** setting ownership and **`chmod`** granting read/execute permissions. Apache2 sends the HTML content to the client's browser via TCP, displaying "Welcome to WS1!" or "Welcome to WS2!". UFW secures the process, and file permissions prevent unauthorized access. The browser screenshots confirm the display. The **`DocumentRoot`** directive is critical for locating the website's files, and the permission commands ensure secure, reliable content delivery.

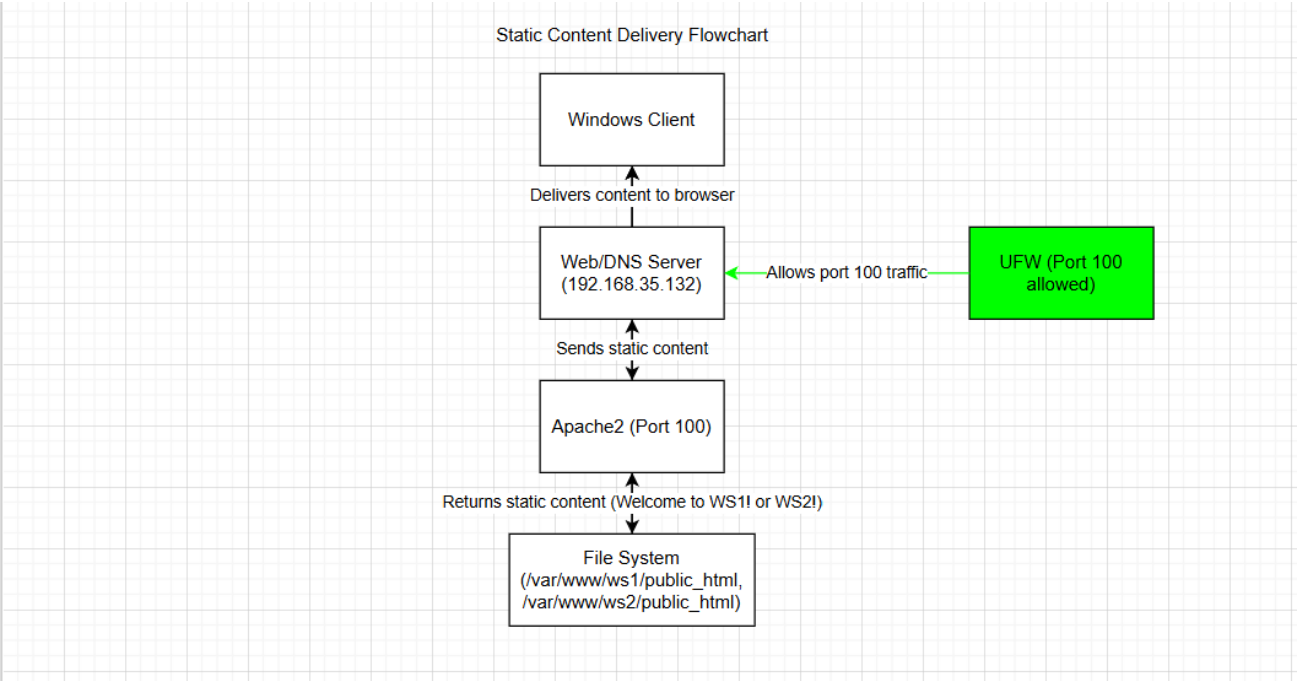

Figure 3:Static Delivery Flowchar

4. **Firewall Security**: UFW on the web/DNS server allows incoming traffic only on ports 53 (DNS) and 100 (HTTP), using iptables to block unauthorized access to Apache2 and BIND9, ensuring secure communication with the client. The firewall configuration is verified with **`sudo ufw status`**, showing allowed ports.

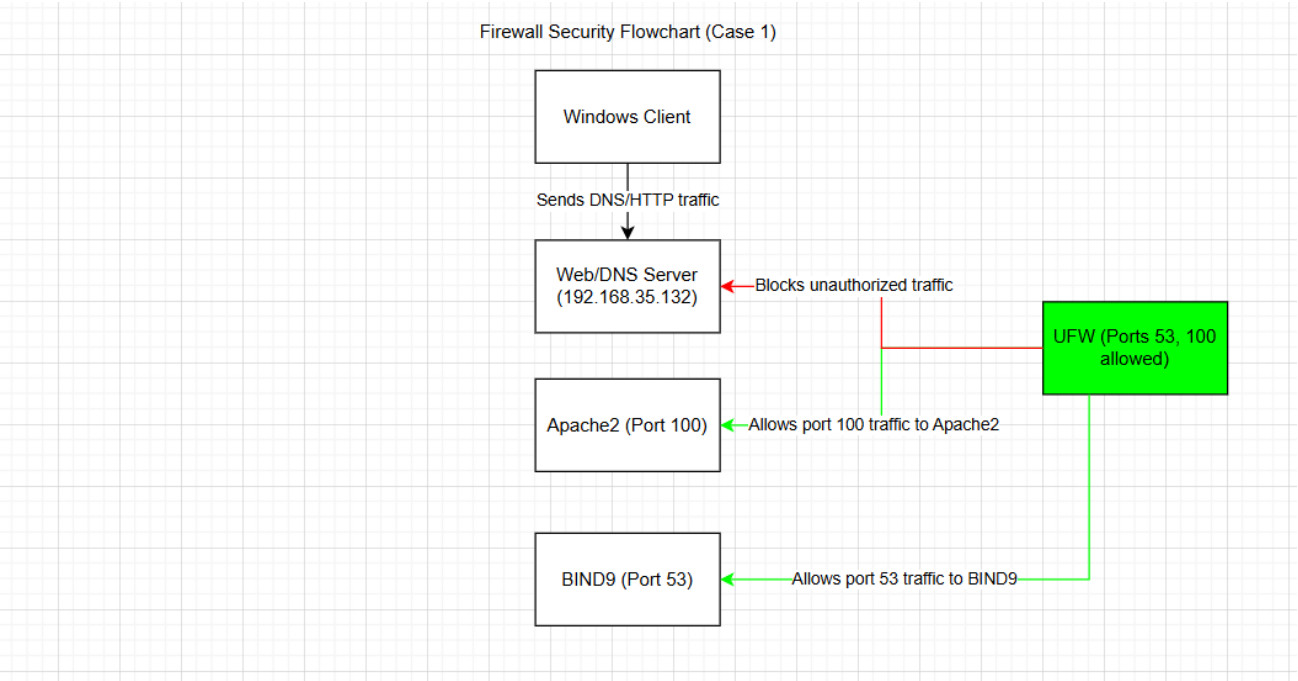

Figure 4: Flrewall Security Flowchar (Case 1)

CASE 2: SERVING DYNAMIC CONTENT VIA MARIADB DATABASE

1. **DNS Query**: Identical to Case 1, the client sends a DNS query to 192.168.35.132:53 for **`ws1.com`** or **`ws2.com`**, and BIND9 resolves it to 192.168.35.132 using **`/etc/bind/named.conf.local`** and zone files (**`/etc/bind/zones/db.ws1.com`** or

db.ws2.com), with **A** records mapping domains to the IP. UFW allows port 53 (**sudo ufw allow 53**). Verification is done with **nslookup ws1.com 192.168.35.132** or **nslookup ws2.com 192.168.35.132**. The **zone** and **A** record commands ensure accurate resolution, directing the HTTP request to Apache2.

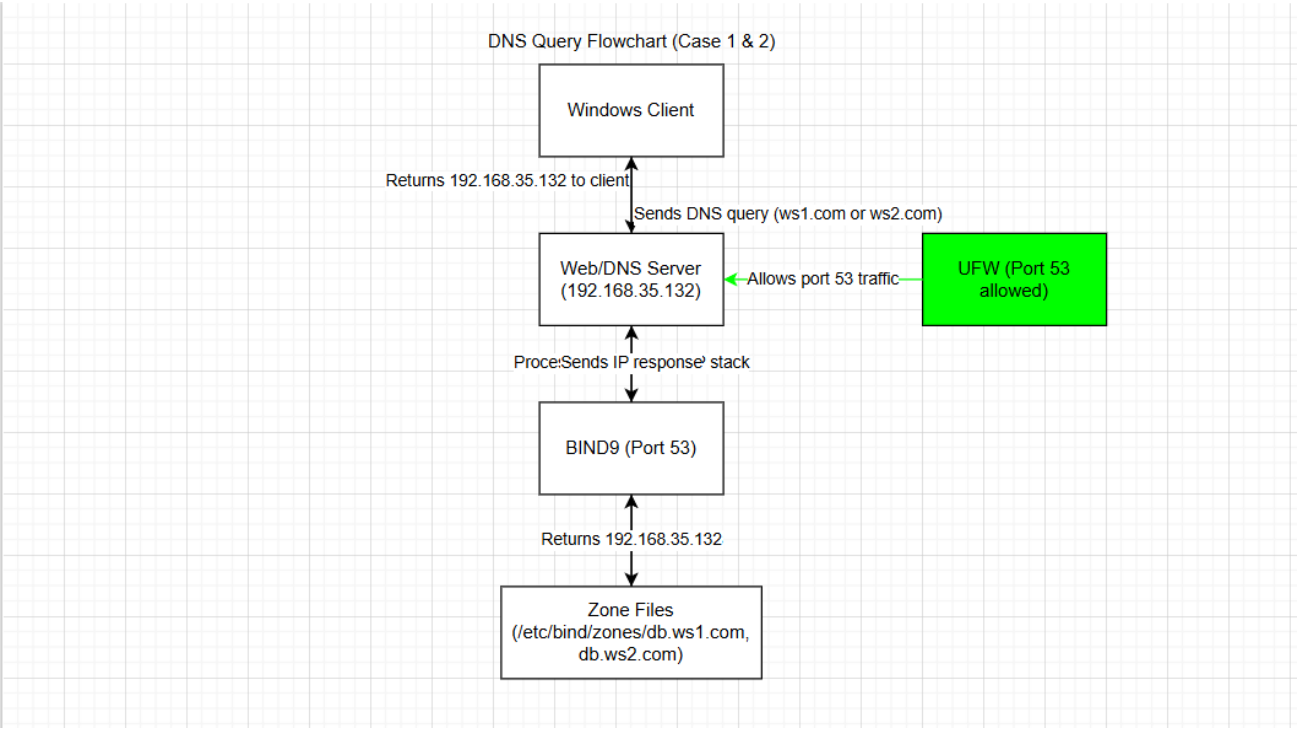

Figure 5: DNS Query Flowchart (Case 1 & 2)

2. **HTTP Request**: As in Case 1, the client sends an HTTP request to **http://ws1.com:100/DVWA/** or **http://ws2.com:100/DVWA**/, with the Host header (**ws1.com** or **ws2.com**). Apache2, listening on port 100 (**Listen 100** in **/etc/apache2/ports.conf**), matches the **Host** header to **ServerName** in **/etc/apache2/sites-available/ws1.com.conf** or **ws2.com.conf**, routing to **/var/www/ws1/public_html/DVWA** or **/var/www/ws2/public_html/DVWA**. The virtual host includes PHP handling:

```
<VirtualHost *:100>
   ServerName ws1.com
   DocumentRoot /var/www/ws1/public_html
   <Directory /var/www/ws1/public_html/DVWA>
      Options Indexes FollowSymLinks
      AllowOverride All
      Require all granted
   </Directory>
   <FilesMatch "\.php$">
      SetHandler application/x-httpd-php
   </FilesMatch>
</VirtualHost>
```

The **ServerName** and **DocumentRoot** directives differentiate requests, and **<FilesMatch "\.php$">** ensures PHP scripts (e.g., **index.php**) are executed. UFW allows port 100 (**sudo ufw allow 100/tcp**).

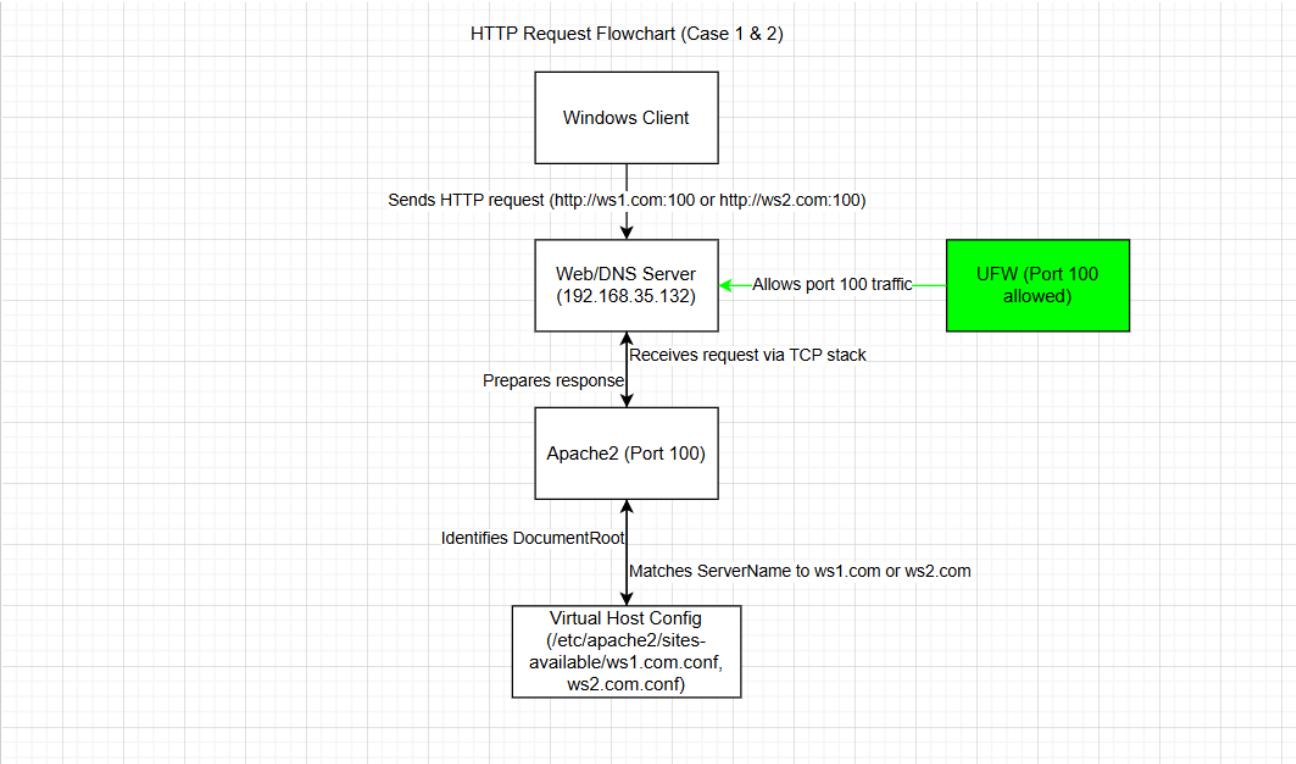

Figure 6: HTTP Request Flowchart (Case 1 & 2)

3. **Database Query**: Apache2 executes PHP scripts in **`/var/www/ws1/public_html/DVWA`** (or **`/var/www/ws2/public_html/DVWA`**), which read **`/var/www/ws1/public_html/DVWA/config/config.inc.php`**:

```
$_DVWA['db_server'] = '192.168.35.133';
$_DVWA['db_database'] = 'dvwa_ws1';
$_DVWA['db_user'] = 'huynhan';
$_DVWA['db_password'] = 'huynhan1812';
```

For **`ws2.com, /var/www/ws2/public_html/DVWA/config/config.inc.php`** specifies **`db_database = 'dvwa_ws2'`**. The **`$_DVWA['db_database']`** parameter ensures the PHP script connects to the correct database (**`dvwa_ws1`** or **`dvwa_ws2`**) on the MariaDB server (192.168.35.133:3306) using the MariaDB client with user huynhan and password huynhan1812. MariaDB, configured with **`/etc/mysql/mariadb.conf.d/50-server.cnf`** (**`bind-address = 0.0.0.0`**), allows remote connections from 192.168.35.132. Databases and user privileges are set via:

```
CREATE DATABASE dvwa_ws1;
CREATE DATABASE dvwa_ws2;
CREATE USER 'huynhan'@'192.168.35.132' IDENTIFIED BY 'huynhan1812';
GRANT ALL PRIVILEGES ON dvwa_ws1.* TO 'huynhan'@'192.168.35.132';
GRANT ALL PRIVILEGES ON dvwa_ws2.* TO 'huynhan'@'192.168.35.132';
FLUSH PRIVILEGES;
```

The **`CREATE DATABASE`** commands establish isolated databases, and **`GRANT`** ensures **`huynhan`** accesses only the specified database from the Web/DNS server, preventing crossover. UFW on the database server allows port 3306 (**`sudo ufw allow 3306/tcp`**). Connectivity is verified with **`mariadb -u huynhan -p -h 192.168.35.133 -e "SHOW DATABASES;"`**. The

config.inc.php file is critical for directing PHP to the correct database, and SQL commands ensure secure, isolated data access.

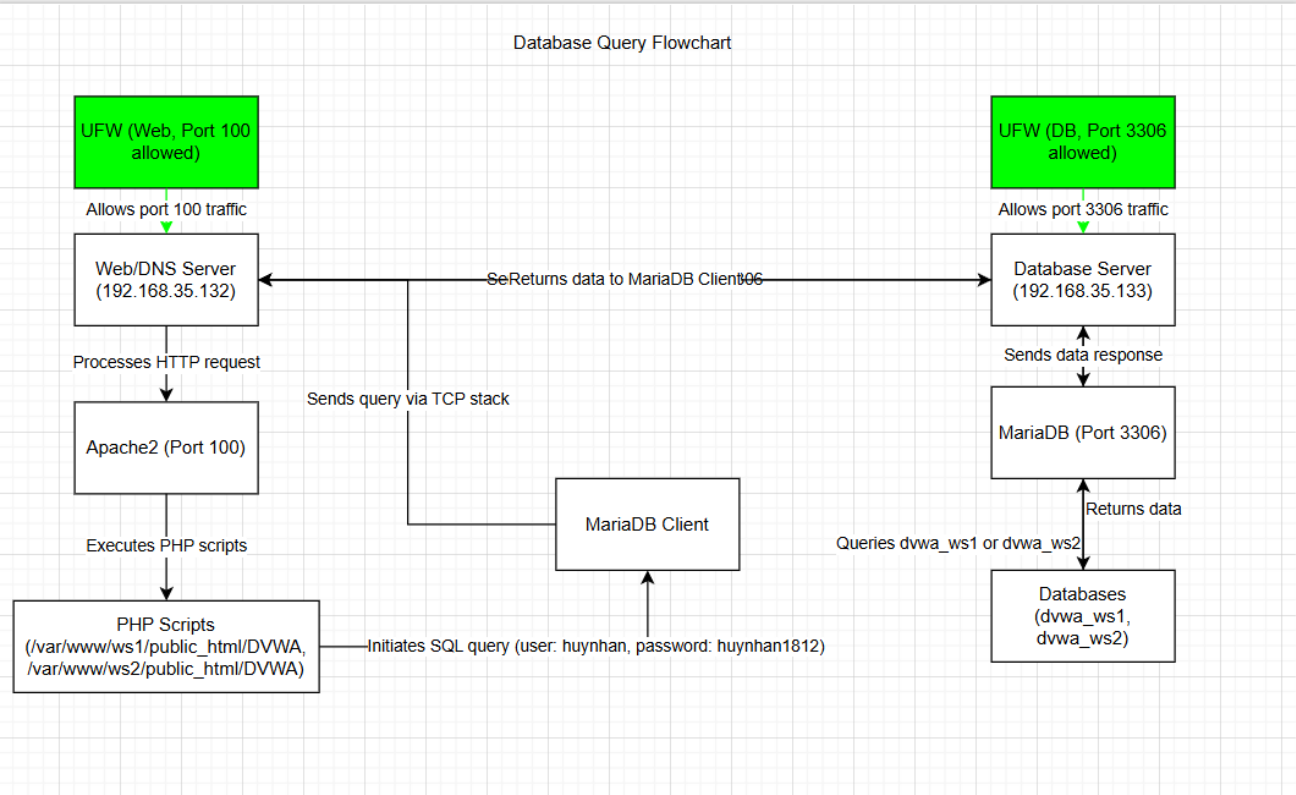

Figure 7: Database Query Flowchart

4. **Dynamic Content Delivery**: PHP processes the database results (e.g., user data for the DVWA login page) and generates dynamic HTML, which Apache2 sends to the client's browser. DVWA files in **/var/www/ws1/public_html/DVWA** (or **/var/www/ws2/public_html/DVWA**) have permissions 755 and owner **www-data**, set by:

```
sudo chown -R www-data:www-data /var/www
sudo chmod -R 755 /var/www
```

These commands ensure Apache2 can access the files securely. The client accesses **http://ws1.com:100/DVWA/setup.php** to initialize **dvwa_ws1**, then **http://ws1.com:100/DVWA/login.php** with **admin/password**, displaying the login page. The same applies for **ws2.com** with **dvwa_ws2**. The **<FilesMatch "\.php$">** directive in the virtual host ensures PHP execution, and **config.inc.php** guarantees data from the correct database.

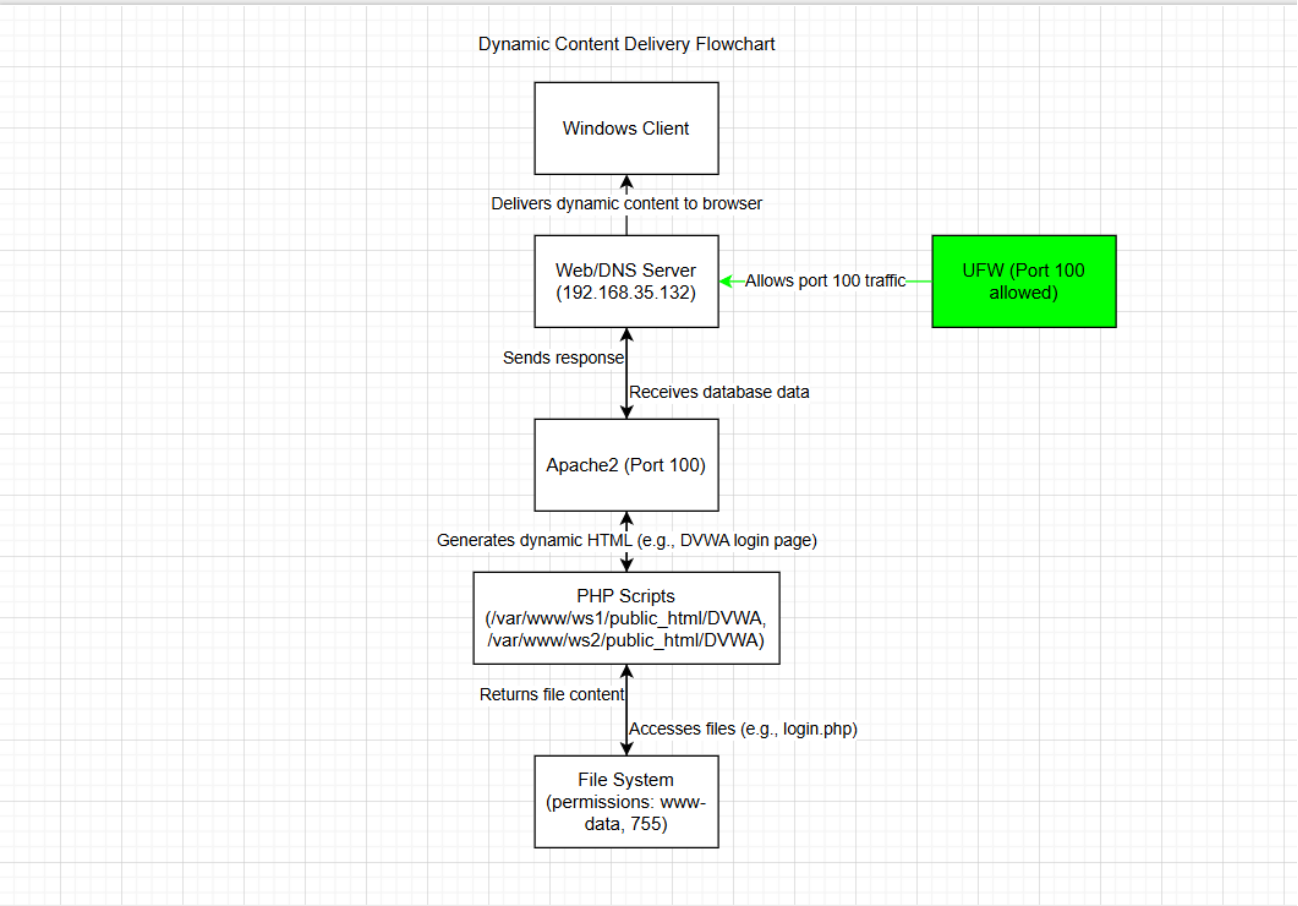

Figure 8: Dynamic Content Delivery Flowchart

5. **Firewall Security**: UFW on the database server allows port 3306 (**`sudo ufw allow 3306/tcp`**), securing MariaDB access for the Web/DNS server (192.168.35.132). The Web/DNS server's UFW allows ports 53 and 100, as in Case 1. The **`ufw allow 3306/tcp`** command is critical for enabling secure database queries, and **`sudo ufw status`** verifies rules on both servers. These commands protect all services, ensuring safe client and web-to-database communication.

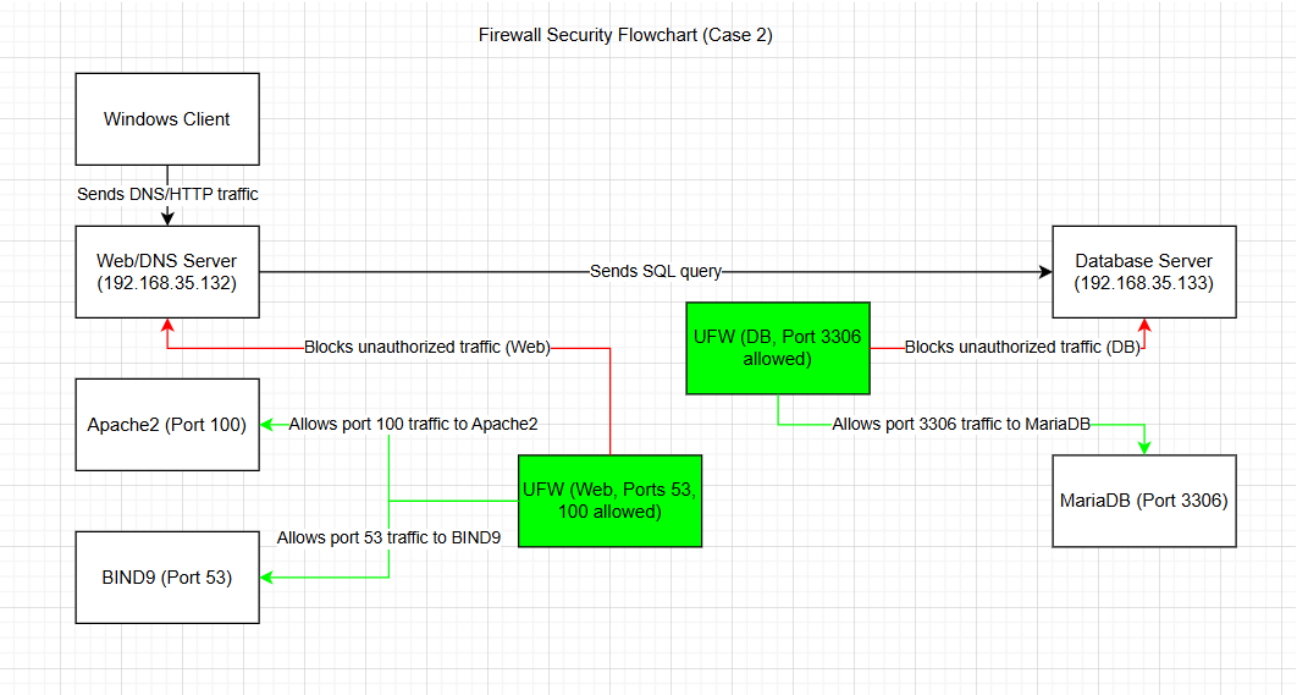

Figure 9: Firewall Security Flowchart(Case 2)

II. IMPLEMENTATION

1. TOPOLOGY

The basic setup topology includes a single Kali Linux VM (192.168.35.132) hosting Apache2 (port 100) and BIND9 (port 53), connected to a Windows client via a NAT network with a gateway/DNS at 192.168.35.2. The advanced setup adds a second Kali Linux VM (192.168.35.133) running MariaDB (port 3306) for dynamic data.

BASIC SETUP TOPOLOGY:

```text
+----------------------+        +------------------+
| Kali Linux Server    |        | Windows Host     |
| (192.168.35.132)     |        | (Client)         |
| - Apache2 (port 100)|<------>| - Browser        |
| - BIND9 (port 53)    |        | - nslookup       |
| - UFW                |        |                  |
+----------------------+        +------------------+
```

ADVANCE SETUP TOPOLOGY:

```text
+--------------------+        +--------------------+        +-----------------+
| Kali Linux Server  |        | Kali Linux Server  |        | Windows Host    |
| (192.168.35.132)   |        | (192.168.35.133)   |        | (Client)        |
| - Apache2 (port 100)|<------>| - MariaDB (port 3306)|<------>| - Browser       |
| - BIND9 (port 53)  |        | - UFW              |        | - nslookup      |
| - UFW              |        |                    |        |                 |
+--------------------+        +--------------------+        +-----------------+
```

Table 1: Components and Specifications

Component	Specification
Web/DNS Server	Kali Linux 2025.2, IP: 192.168.35.132, Apache2 (port 100), BIND9 (port 53), UFW
Database Server	Kali Linux 2025.2, IP: 192.168.35.133, MariaDB (port 3306), UFW
Windows Client	Windows host, browser, nslookup

Component	Specification
Network	NAT mode, DHCP, gateway/DNS: 192.168.35.2, network: 192.168.35.0/24

2. INSTALLATION

The installation process on the web/DNS server (192.168.35.132) and database server (192.168.35.133) ensures all components function for both basic and advanced setups, covering system updates, firewall, web, DNS, PHP, MariaDB, and DVWA. Each component's installation is described in a cohesive paragraph, including commands, descriptions, verifications, and interactions, with additional command lists for DVWA deployment.

The system update on both the web/DNS server (192.168.35.132) and database server (192.168.35.133) ensures a stable environment for service installations. The command **`sudo apt update && sudo apt upgrade -y`** updates package lists and installs the latest versions of all packages, ensuring compatibility and security for Apache2, BIND9, and MariaDB. This step is critical to prevent dependency issues and vulnerabilities, providing a robust foundation for subsequent installations. Verification with **`lsb_release -a`** confirms Kali Linux 2025.2 on both servers. The system update interacts with all components by ensuring their dependencies are current, enabling Apache2 to serve content, BIND9 to resolve domains, and MariaDB to manage databases without conflicts.

System Update Commands:

- sudo apt update && sudo apt upgrade -y
- lbs_reslease -a

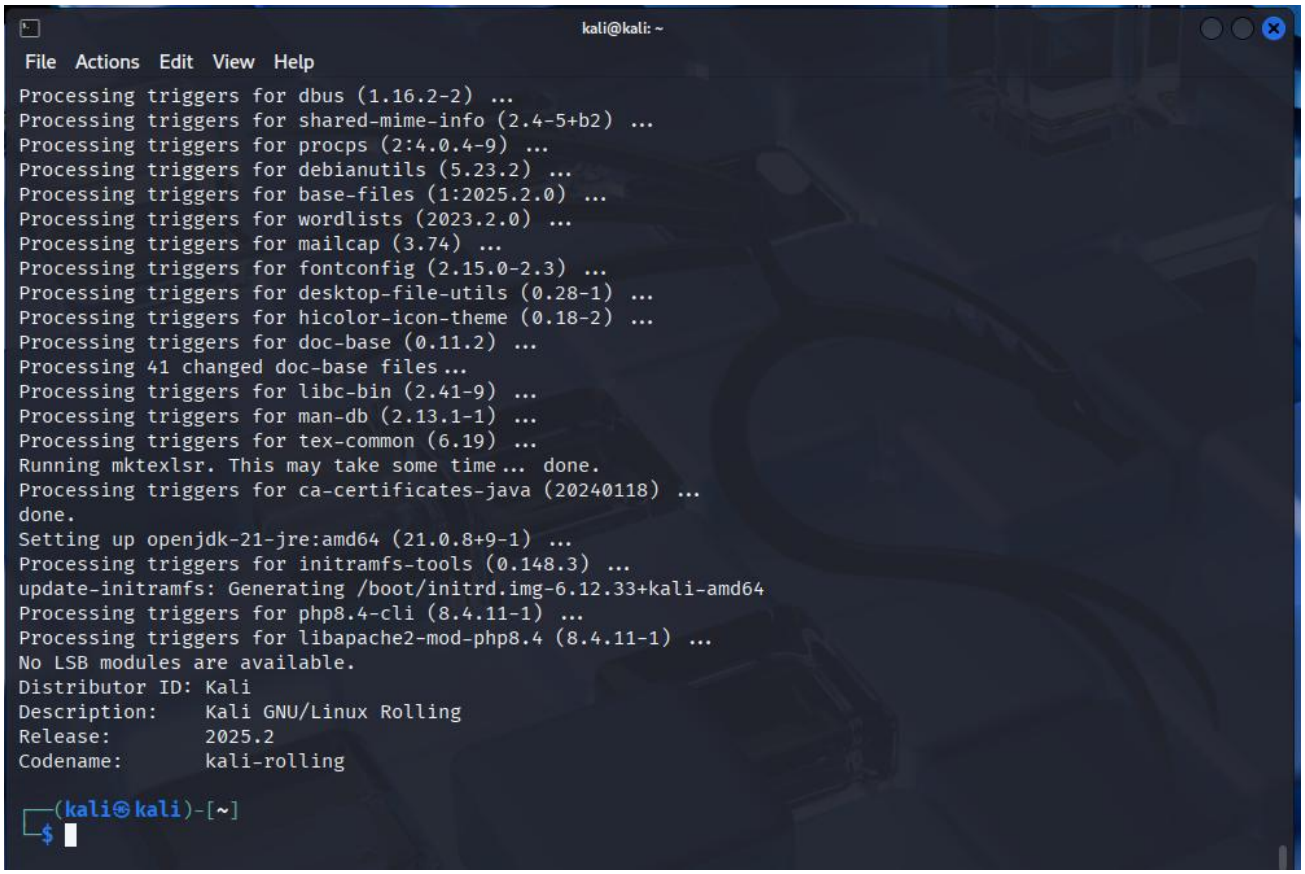

Figure 10: Kali Linux System and Package Verification

The UFW installation on both servers secures network traffic for all services. The command **`sudo apt install ufw -y`** installs the Uncomplicated Firewall, which will be configured to allow specific ports (100 and 53 for web/DNS, 3306 for database). This ensures protection against unauthorized access while allowing legitimate traffic to Apache2, BIND9, and MariaDB. Verification with **`ufw --version`** confirms the firewall's installation and version. UFW interacts with all services by filtering traffic, ensuring secure communication between the client, web server, and database server, and protecting against external threats.

UFW Installation Commands:

- sudo apt install ufw -y
- ufw --version

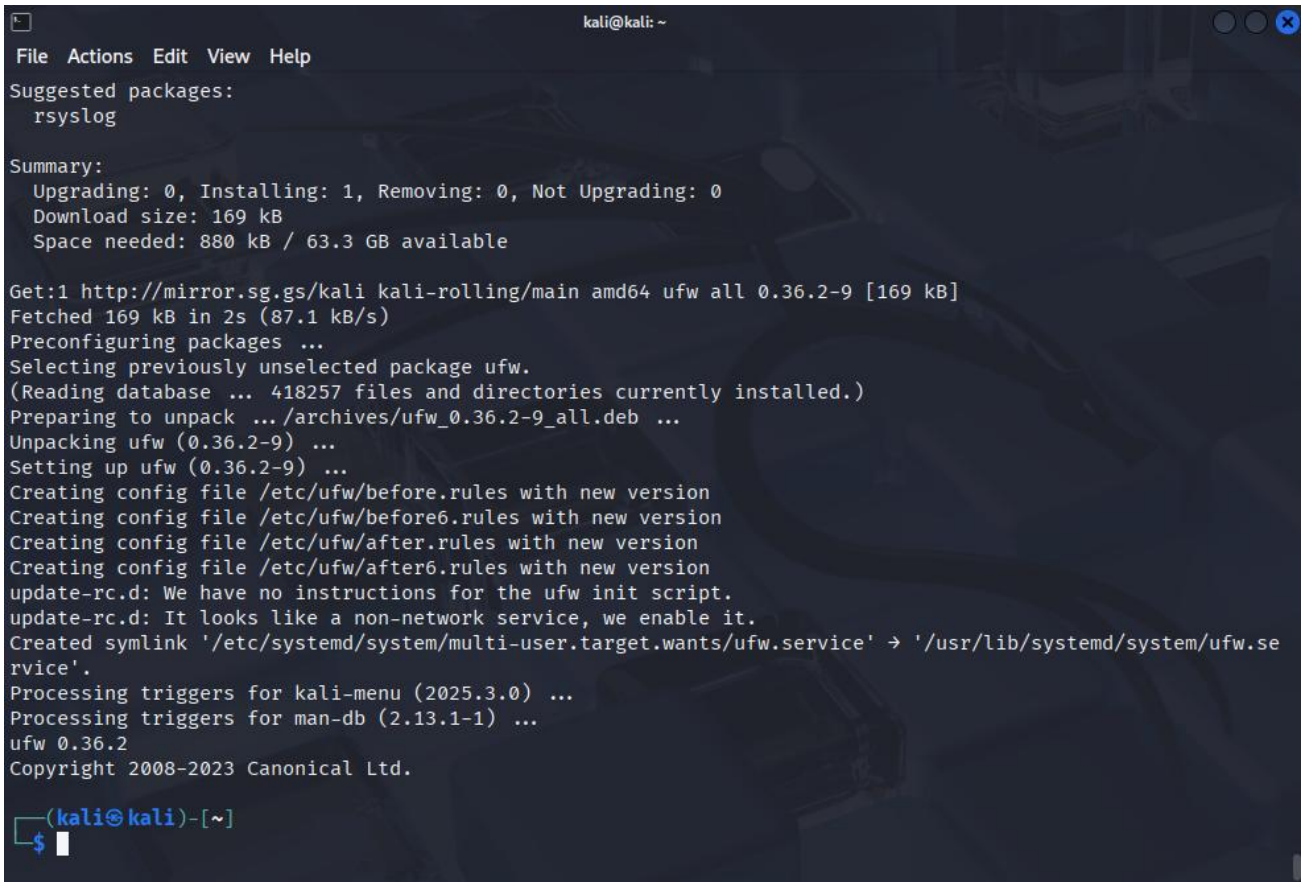

Figure 11: UFW Firewall Configuration for Web Server

The Apache2 installation on the web/DNS server (192.168.35.132) enables web content delivery for **`ws1.com`** and **`ws2.com`**. The command **`sudo apt install apache2 -y`** installs Apache2, followed by **`sudo systemctl start apache2`** to start the service and **`sudo systemctl enable apache2`** to ensure it runs at boot. This setup supports HTTP requests on port 100, hosting both static and dynamic content. Verification with **`apache2 -v`** confirms the Apache2 version, and **`sudo systemctl status apache2`** ensures the service is active. Apache2 interacts with BIND9 by serving content for resolved domains and with the MariaDB client in the advanced setup to fetch dynamic data for DVWA, forming the core of the web system.

Apache2 Installation Commands:

- sudo apt install apache2 -y
- sudo systemctl start apache2
- sudo systemctl enable apache2
- apache2 -v
- sudo systemctl status apache2

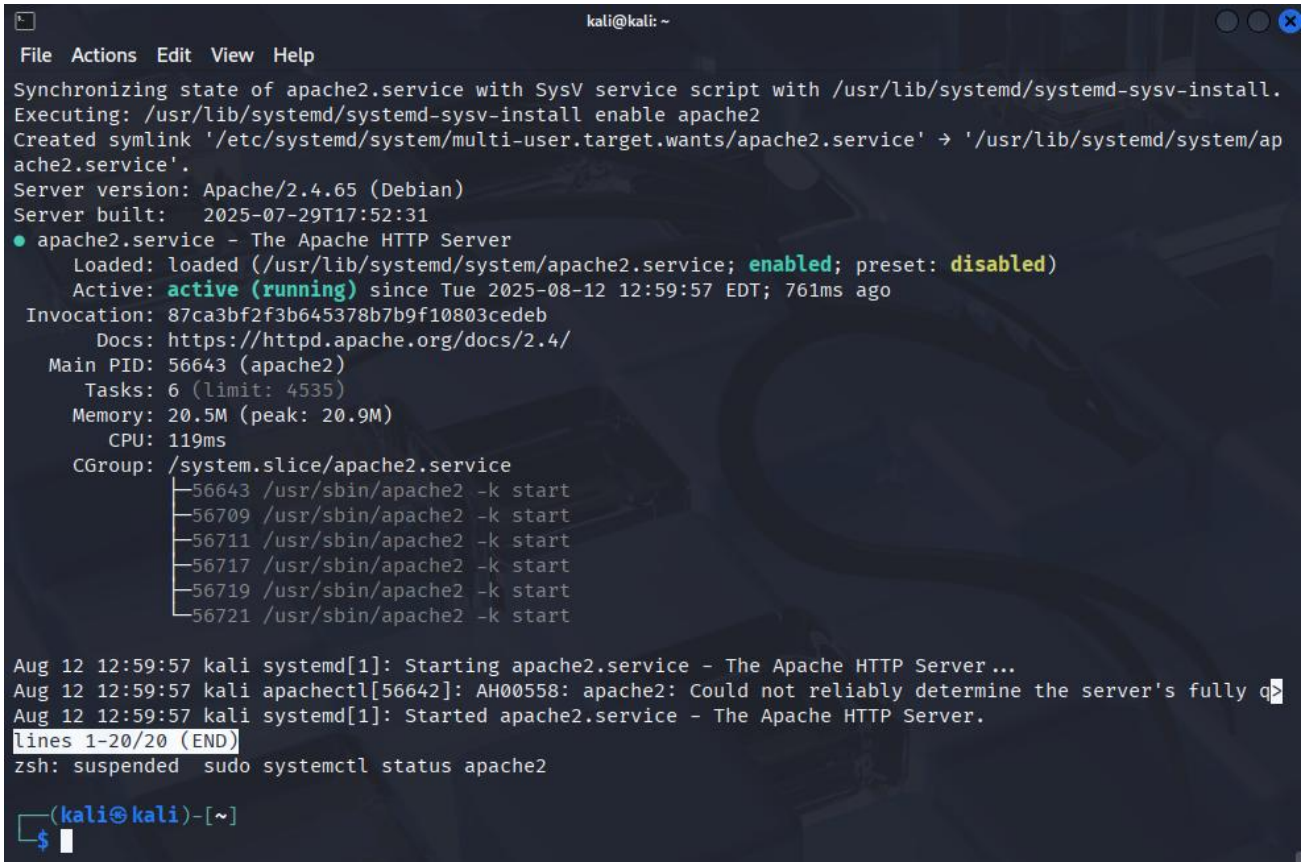

Figure 12: Apache2 and BIND9 Installation & Service Status

The BIND9 installation on the web/DNS server (192.168.35.132) enables domain name resolution for ws1.com and **ws2.com**. The command **sudo apt install bind9 bind9utils bind9-doc -y** installs BIND9 and its utilities, followed by **sudo systemctl start named** to start the service and **sudo systemctl enable named** to ensure it runs at boot. This setup resolves domains to 192.168.35.132 on port 53, directing clients to Apache2. Verification with **named -v** confirms the BIND9 version, and **sudo systemctl status named** ensures the service is active. BIND9 interacts with Apache2 by providing IP addresses for client requests, enabling accurate routing to virtual hosts, and indirectly supports database queries in the advanced setup.

BIND9 Installation Commands:

- sudo apt install bind9 bind9utils bind9-doc -y
- sudo systemctl start named
- sudo systemctl enable named
- named -v
- sudo systemctl status named

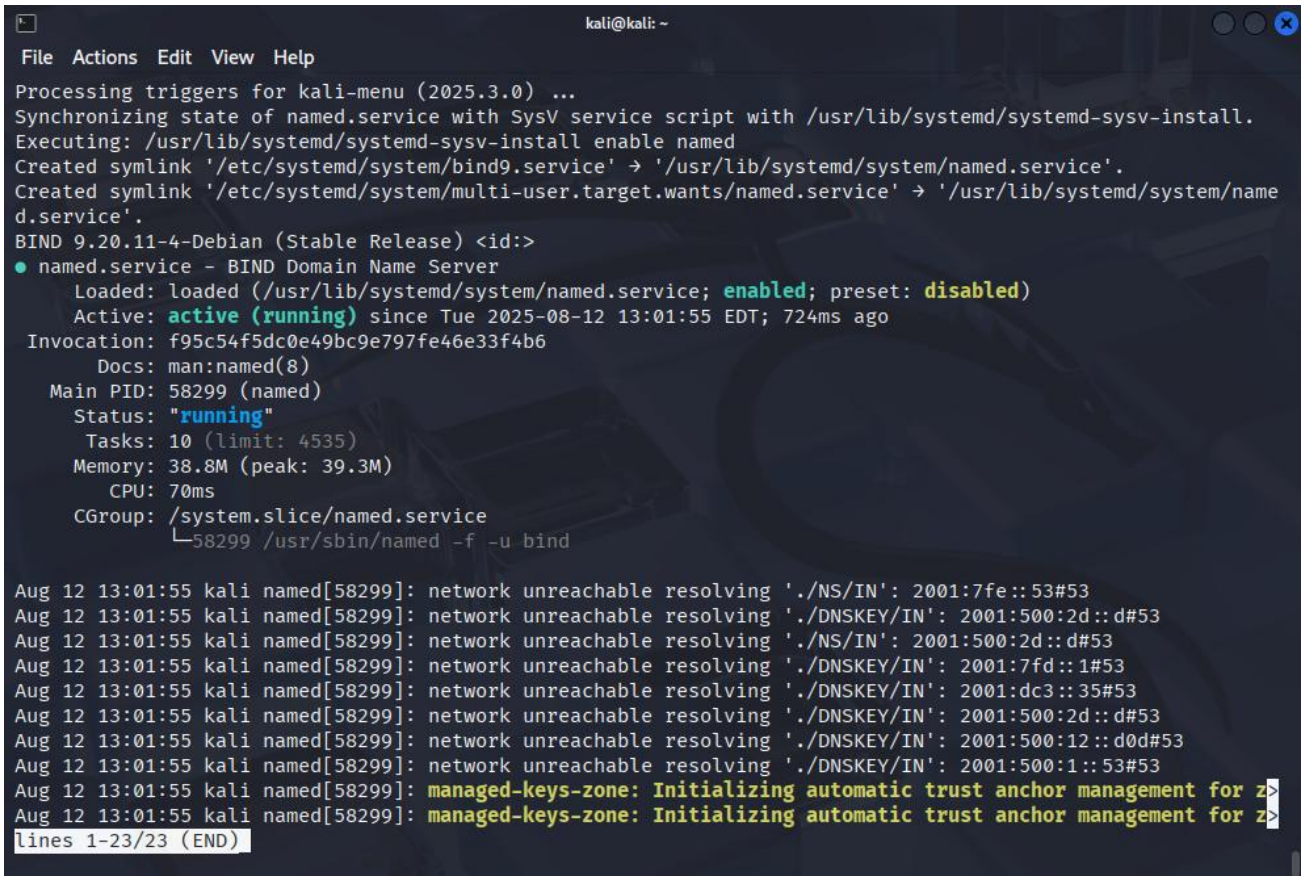

Figure 13: Apache2 and BIND9 Installation & Service Status

The PHP and MariaDB client installation on the web/DNS server (192.168.35.132) supports dynamic content for both **ws1.com** and **ws2.com**. The command **sudo apt install php php-mysql php-gd libapache2-mod-php8.4 mariadb-client git -y** installs PHP for processing dynamic scripts, the MariaDB client for connecting to the database server (192.168.35.133), and Git for cloning DVWA. To support DVWA's features like File Inclusion, **sudo nano /etc/php/8.4/apache2/php.ini** is used to set **allow_url_include = On**. The PHP module and **mod_rewrite** are enabled with **sudo a2enmod php8.4** and **sudo a2enmod rewrite**, followed by **sudo systemctl restart apache2**. Verification with **php -v** confirms the PHP version, and **mariadb --version** ensures the MariaDB client is installed.

PHP and MariaDB Client Installation Commands:

- sudo apt install php php-mysql php-gd libapache2-mod-php8.4 mariadb-client git -y
- sudo nano /etc/php/8.4/apache2/php.ini (set allow_url_include = On)

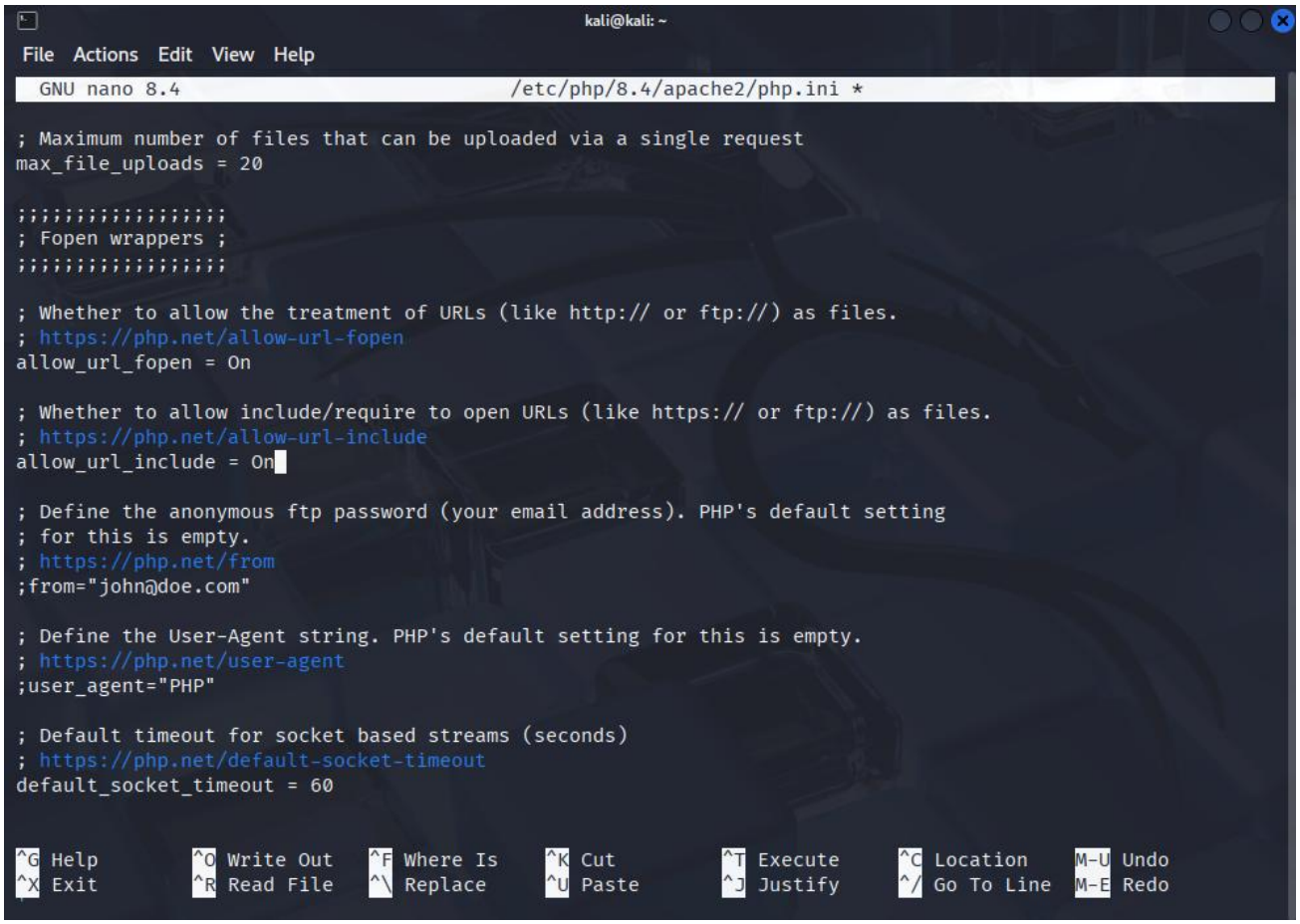

Figure 14: PHP Configuration

- sudo a2enmod php8.4
- sudo a2enmod rewrite
- sudo systemctl restart apache2
- php -v
- mariadb --version

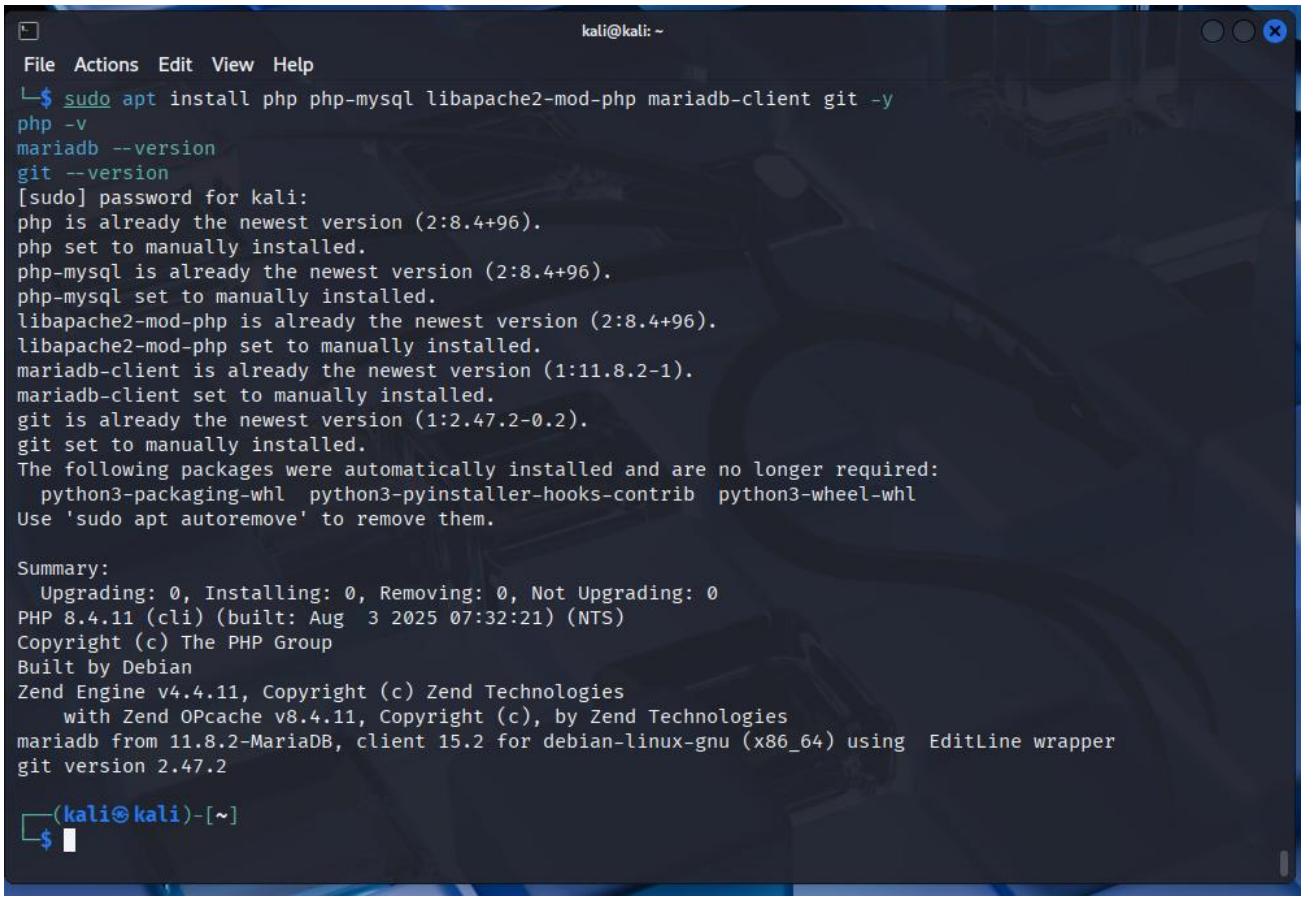

Figure 15: PHP and MariaDB Client Installation

The MariaDB installation on the database server (192.168.35.133) supports dynamic data for **`ws1.com`** and **`ws2.com`**. The command sudo apt install mariadb-server -y installs MariaDB, followed by **`sudo systemctl start mariadb`** to start the service and **`sudo systemctl enable mariadb`** to ensure it runs at boot. The command **`sudo mariadb-secure-installation`** secures the installation by setting a root password (**`huynhan1812`**), removing anonymous users, disabling remote root login, and removing the test database. Verification with **`mariadb --version`** confirms the MariaDB version, and **`sudo systemctl status mariadb`** ensures the service is active. MariaDB interacts with the web server's MariaDB client, processing SQL queries to provide data for DVWA, secured by authentication and UFW.

MariaDB Installation Commands:

- sudo apt install mariadb-server -y
- sudo systemctl start mariadb
- sudo systemctl enable mariadb
- sudo mariadb-secure-installation
- mariadb --version
- sudo systemctl status mariadb

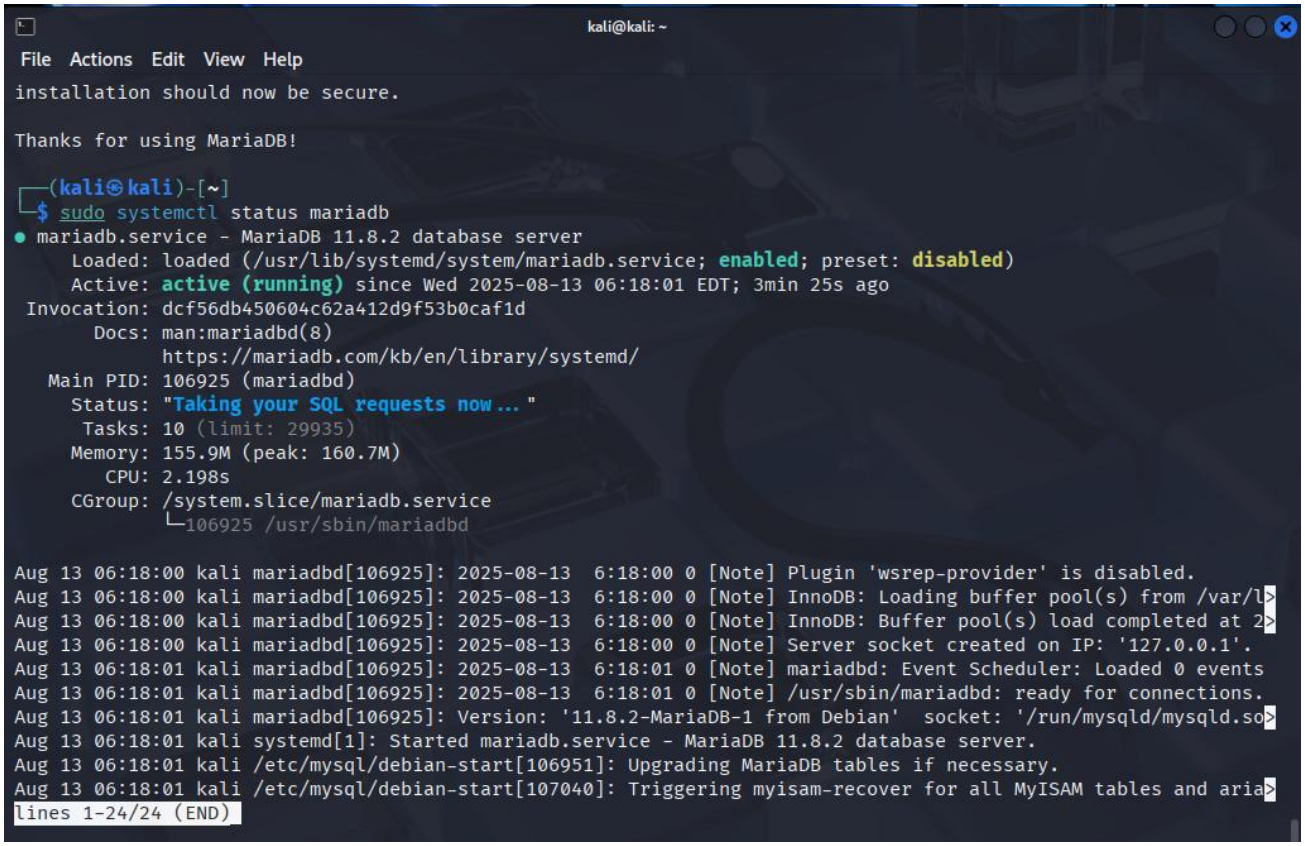

Figure 16: MariaDB Installation on Database Server

The DVWA installation on the web/DNS server (192.168.35.132) deploys the dynamic web application for **ws1.com** and **ws2.com**. The commands s**udo git clone <https://github.com/digininja/DVWA.git> /var/www/ws1/public_html/DVWA** and **sudo git clone <https://github.com/digininja/DVWA.git> /var/www/ws2/public_html/DVWA** clone DVWA for both websites. Permissions are set with **sudo chown -R www-data:www-data /var/www/ws1/public_html /var/www/ws2/public_html** and **sudo chmod -R 755 /var/www/ws1/public_html /var/www/ws2/public_html** to allow Apache2 access. Verification with **ls -l /var/www/ws1/public_html/DVWA** confirms the files are correctly installed, including **setup.php** and **config/config.inc.php**. DVWA interacts with Apache2 to serve dynamic content and with MariaDB via PHP to fetch database data, enabling the advanced setup's functionality.

DVWA Installation Commands:

- sudo git clone <https://github.com/digininja/DVWA.git> /var/www/ws1/public_html/DVWA
- sudo git clone <https://github.com/digininja/DVWA.git> /var/www/ws2/public_html/DVWA
- sudo chown -R www-data:www-data /var/www/ws1/public_html /var/www/ws2/public_html
- sudo chmod -R 755 /var/www/ws1/public_html /var/www/ws2/public_html
- ls -l /var/www/ws1/public_html/DVWA
- ls -l /var/www/ws2/public_html/DVWA

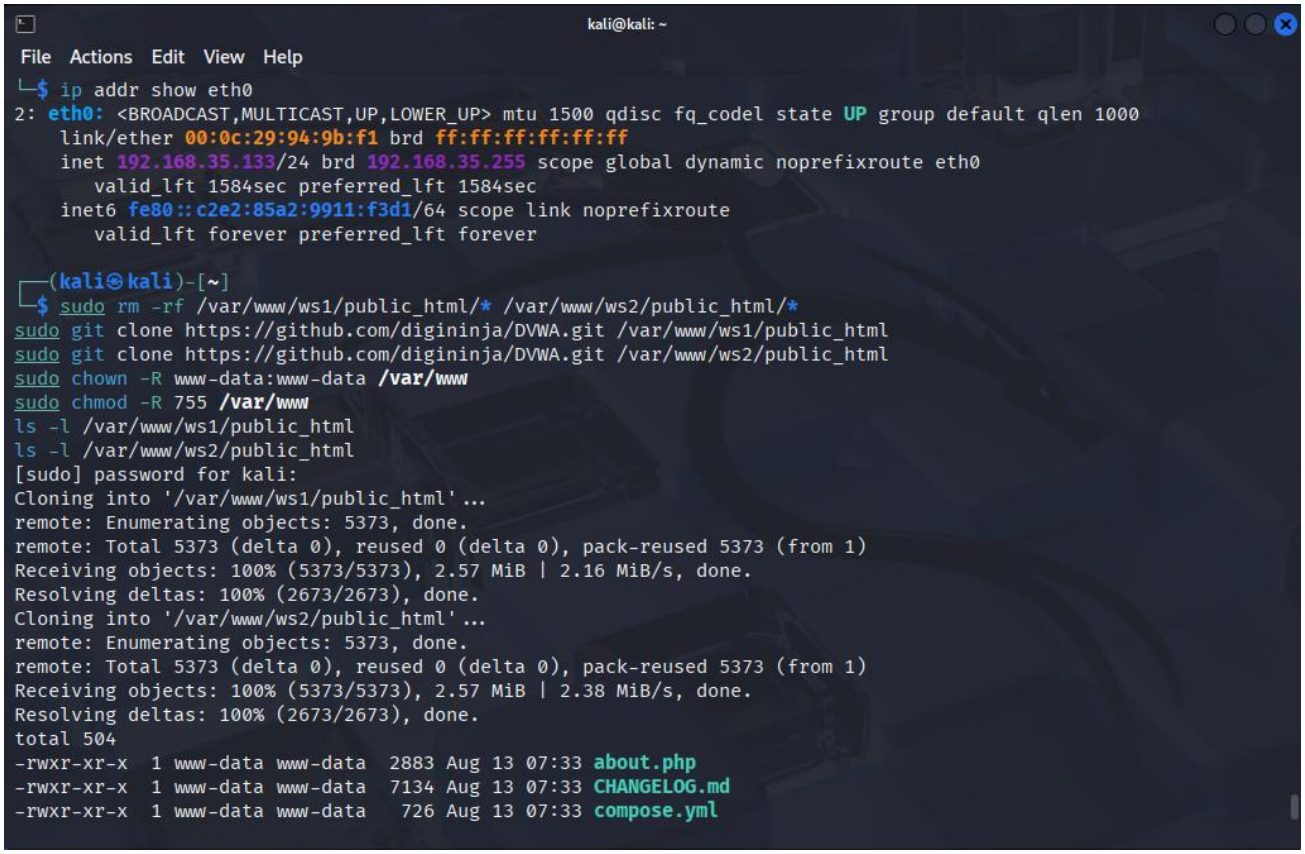

Figure 17: Git Clone DVWA on Database Server

3. CONFIGURATION

1. BASIC WEB DEPLOYMENT

Web Overview:

Aspect	Details
Domains	**`ws1.com`** (static HTML, dynamic in advanced setup), **`ws2.com`** (static HTML), hosted on web/DNS server (**`192.168.35.132`**). Client sends DNS queries to BIND9, resolving to **`192.168.35.132`**, enabling Apache2 to serve content.

Aspect	Details
Server	Web/DNS server (**192.168.35.132**) runs Apache2 (port 100) and BIND9 (port 53). BIND9 resolves domains, directing client HTTP requests to Apache2, which matches **ServerName** to serve content from **/var/www/ws1/public_html** or **/var/www/ws2/public_html**.
Apache2	Hosts virtual hosts on port 100, with **ServerName ws1.com** and **ws2.com** in **/etc/apache2/sites-available/ws1.com.conf** and **ws2.com.conf**, serving content from **/var/www/ws1/public_html** and **/var/www/ws2/public_html**. Apache2 uses **www-data** permissions and interacts with BIND9's resolved IPs, secured by UFW.
BIND9	Resolves **ws1.com** and **ws2.com** to **192.168.35.132** via **/etc/bind/zones/db.ws1.com** and **db.ws2.com**. The **ServerName** in Apache2's virtual hosts maps to IPs in zone files, enabling client requests to reach Apache2, secured by UFW on port 53.
UFW	Allows ports 100 (HTTP) and 53 (DNS), using iptables to protect Apache2 and BIND9. UFW ensures only client DNS queries and HTTP requests reach services, securing communication.

Network Configuration: The command **ip addr show eth0** confirms the Web/DNS server's IP as **192.168.35.132**, ensuring it can communicate within the NAT network. The command **cat /etc/resolv.conf** verifies **nameserver 192.168.35.2**, enabling package downloads and external domain resolution. Connectivity is tested with **ping -c 4 192.168.35.2** (gateway) and **ping -c 4 8.8.8.8** (public internet). If connectivity fails, **sudo nano /etc/resolv.conf adds nameserver 8.8.8.8 and nameserver 1.1.1.1**. This establishes a reliable network foundation for BIND9 and Apache2 operations.

Network Configuration Commands:

- ip addr show eth0
- sudo nano /etc/resolv.conf (add nameserver 8.8.8.8, nameserver 1.1.1.1)
- cat /etc/resolv.conf

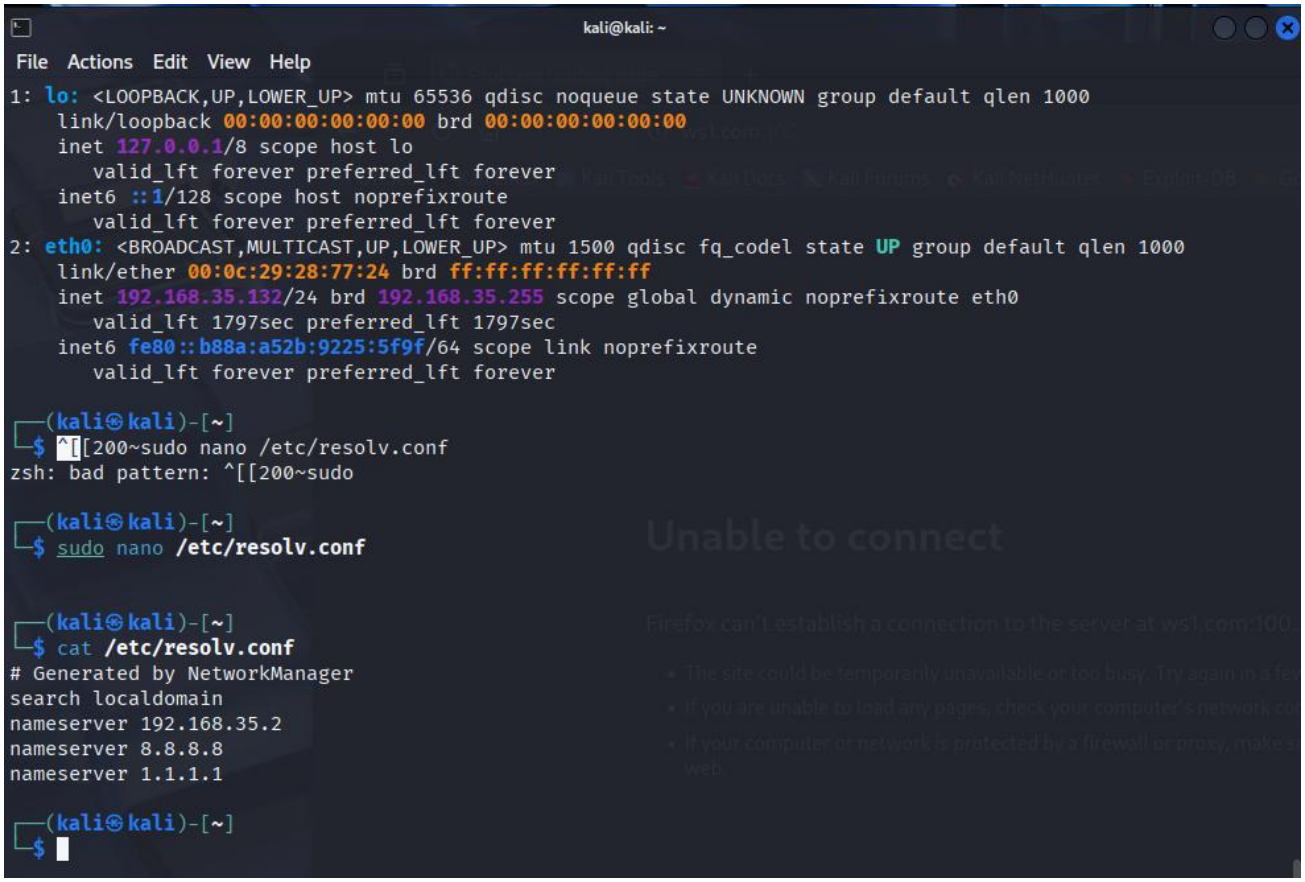

Figure 18: Default Network and DNS Settings

- ping -c 4 192.168.35.2
- ping -c 4 8.8.8.8

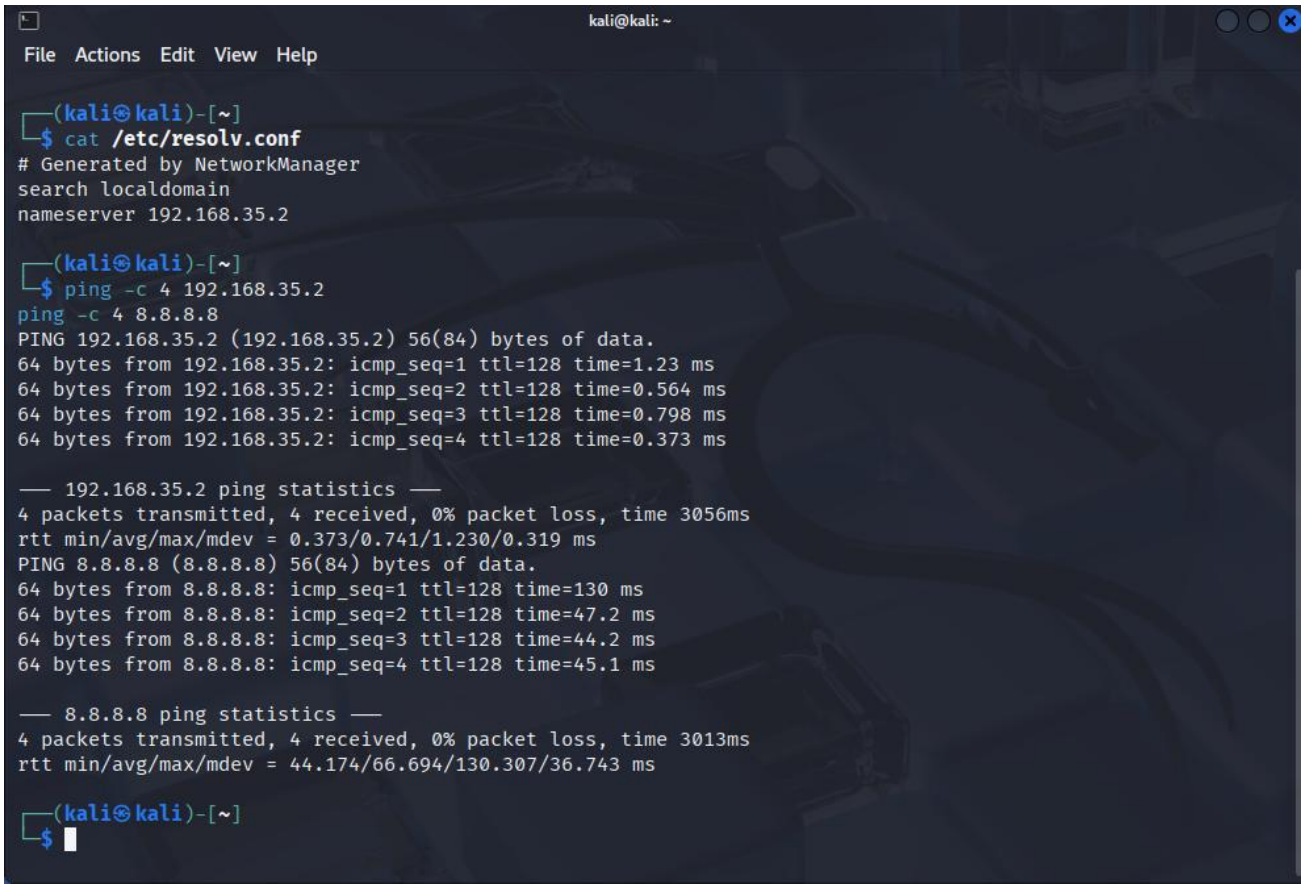

Figure 19: Connectivity Test (Gateway & Public IP)

Apache2 Configuration: To avoid port conflicts, Apache2 is configured to listen on port 100 using **`sudo nano /etc/apache2/ports.conf`** (set **`Listen 100`**). Virtual hosts for **`ws1.com`** and **`ws2.com`** are defined in **`/etc/apache2/sites-available/ws1.com.conf`**:

```
<VirtualHost *:100>
    ServerAdmin admin@ws1.com
    ServerName ws1.com
    ServerAlias www.ws1.com
    DocumentRoot /var/www/ws1/public_html
    ErrorLog ${APACHE_LOG_DIR}/ws1-error.log
    CustomLog ${APACHE_LOG_DIR}/ws1-access.log combined
</VirtualHost>
```

and similarly for **`ws2.com.conf`** . Directories are created with **`sudo mkdir -p /var/www/ws1/public_html /var/www/ws2/public_html`**, and static HTML files are added using **`echo "<h1>Welcome to WS1!</h1>" | sudo tee /var/www/ws1/public_html/index.html`** and similarly for **`ws2.com`**. Permissions are set with **`sudo chown -R www-data:www-data /var/www`** and **`sudo chmod -R 755 /var/www`** to ensure Apache2 can access the files. Sites are enabled with **`sudo a2ensite ws1.com.conf ws2.com.conf`**, and Apache2 is restarted with **`sudo systemctl restart apache2`**. This configuration allows Apache2 to serve static content efficiently.

Apache2 Configuration Commands:

- sudo nano /etc/apache2/ports.conf (set Listen 100)

```
kali@kali: ~
File  Actions  Edit  View  Help
Aug 12 13:01:55 kali named[58299]: network unreachable resolving './DNSKEY/IN': 2001:500:12::d0d#53
Aug 12 13:01:55 kali named[58299]: network unreachable resolving './DNSKEY/IN': 2001:500:1::53#53
Aug 12 13:01:55 kali named[58299]: managed-keys-zone: Initializing automatic trust anchor management for z>
Aug 12 13:01:55 kali named[58299]: managed-keys-zone: Initializing automatic trust anchor management for z>
lines 1-23/23 (END)
zsh: suspended  sudo systemctl status named

┌──(kali㉿kali)-[~]
└─$ sudo nano /etc/apache2/ports.conf

┌──(kali㉿kali)-[~]
└─$

┌──(kali㉿kali)-[~]
└─$ cat /etc/apache2/ports.conf
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 100

<IfModule ssl_module>
        Listen 443
</IfModule>

<IfModule mod_gnutls.c>
        Listen 443
</IfModule>

┌──(kali㉿kali)-[~]
└─$
```

Figure 20: Port Configuration for Listen 100

- sudo nano /etc/apache2/sites-available/ws1.com.conf (configure virtual host)

```
kali@kali: ~
File  Actions  Edit  View  Help
  GNU nano 8.4                    /etc/apache2/sites-available/ws1.com.conf
<VirtualHost *:100>
    ServerAdmin admin@ws1.com
    ServerName ws1.com
    ServerAlias www.ws1.com
    DocumentRoot /var/www/ws1/public_html
    ErrorLog ${APACHE_LOG_DIR}/ws1-error.log
    CustomLog ${APACHE_LOG_DIR}/ws1-access.log combined
</VirtualHost>

                                  [ Wrote 8 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Figure 21: Configuration virtual host for ws1.com on Apache

- sudo nano /etc/apache2/sites-available/ws2.com.conf (configure virtual host)

Figure 22: Configuration virtual host for ws2.com on Apache

- =sudo mkdir -p /var/www/ws1/public_html /var/www/ws2/public_html
- echo "<h1>Welcome to WS1!</h1>" | sudo tee /var/www/ws1/public_html/index.html
- echo "<h1>Welcome to WS2!</h1>" | sudo tee /var/www/ws2/public_html/index.html

```
kali@kali: ~
File  Actions  Edit  View  Help
zsh: event not found: </h1>

┌──(kali㉿kali)-[~]
└─$ echo "<h1>Welcome to WS1!</h1>" | sudo tee /var/www/ws1/public_html/index.html

zsh: event not found: </h1>

┌──(kali㉿kali)-[~]
└─$ echo "<h1>Welcome to WS1 | sudo tee /var/www/ws1/public_html/index.html

dquote> echo "<h1>Welcome to WS1 | sudo tee /var/www/ws1/public_html/index.html

zsh: no such file or directory: h1

┌──(kali㉿kali)-[~]
└─$

┌──(kali㉿kali)-[~]
└─$ echo "<h1>Welcome to WS1</h1>" | sudo tee /var/www/ws1/public_html/index.html
echo "<h1>Welcome to WS2</h1>" | sudo tee /var/www/ws2/public_html/index.html
sudo a2ensite ws1.com.conf ws2.com.conf
sudo systemctl restart apache2
<h1>Welcome to WS1</h1>
<h1>Welcome to WS2</h1>
Enabling site ws1.com.
Enabling site ws2.com.
To activate the new configuration, you need to run:
  systemctl reload apache2

┌──(kali㉿kali)-[~]
└─$
```

Figure 23: Static Page for Basic Setup

- sudo chown -R www-data:www-data /var/www
- sudo chmod -R 755 /var/www
- sudo a2ensite ws1.com.conf ws2.com.conf
- sudo systemctl restart apache2

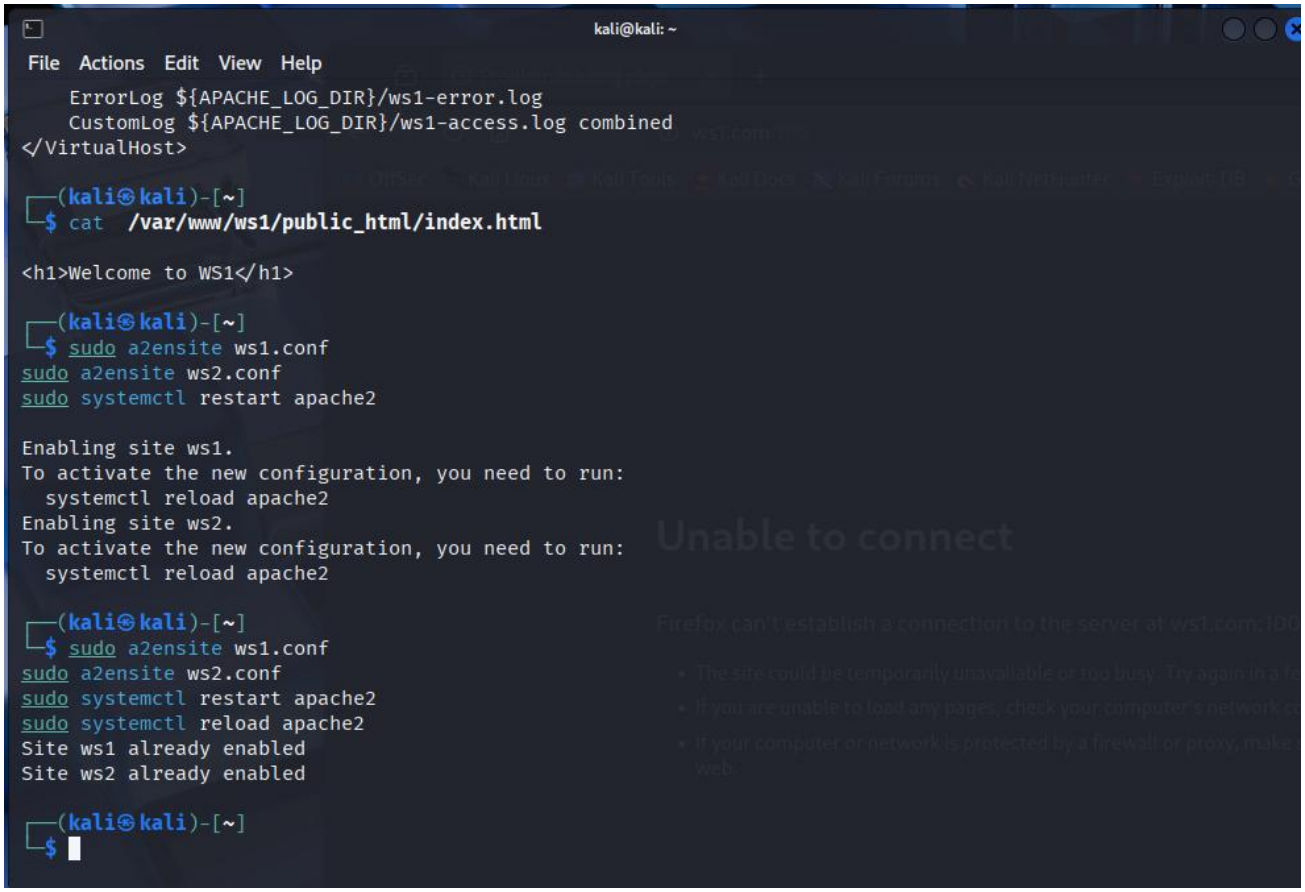

Figure 24: Reload and Restart for Static Page

BIND9 Configuration: BIND9 is configured to resolve **`ws1.com`** and **`ws2.com`** to **`192.168.35.132`**. The command **`sudo nano /etc/bind/named.conf.local`** defines the zones:

```
zone "ws1.com" {
    type master;
    file "/etc/bind/zones/db.ws1.com";
};
zone "ws2.com" {
    type master;
    file "/etc/bind/zones/db.ws2.com";
};
```

Zone files are created with **`sudo nano /etc/bind/zones/db.ws1.com`**:

```
$TTL    604800
@       IN      SOA     ns1.ws1.com. admin.ws1.com. (
                              1         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
@       IN      NS      ns1.ws1.com.
ns1     IN      A       192.168.35.132
@       IN      A       192.168.35.132
www     IN      A       192.168.35.132
```

and similarly for **`db.ws2.com`**. Permissions are set with **`sudo chown -R bind:bind /etc/bind/zones`** and **`sudo chmod -R 755 /etc/bind/zones`**. Zone files are verified with **`sudo named-checkzone ws1.com /etc/bind/zones/db.ws1.com`** and **`sudo named-checkzone ws2.com /etc/bind/zones/db.ws2.com`**, and BIND9 is restarted with **`sudo systemctl restart named`**. BIND9 enables client requests to reach Apache2.

BIND9 Configuration Commands:

- sudo nano /etc/bind/named.conf.local (define zones)

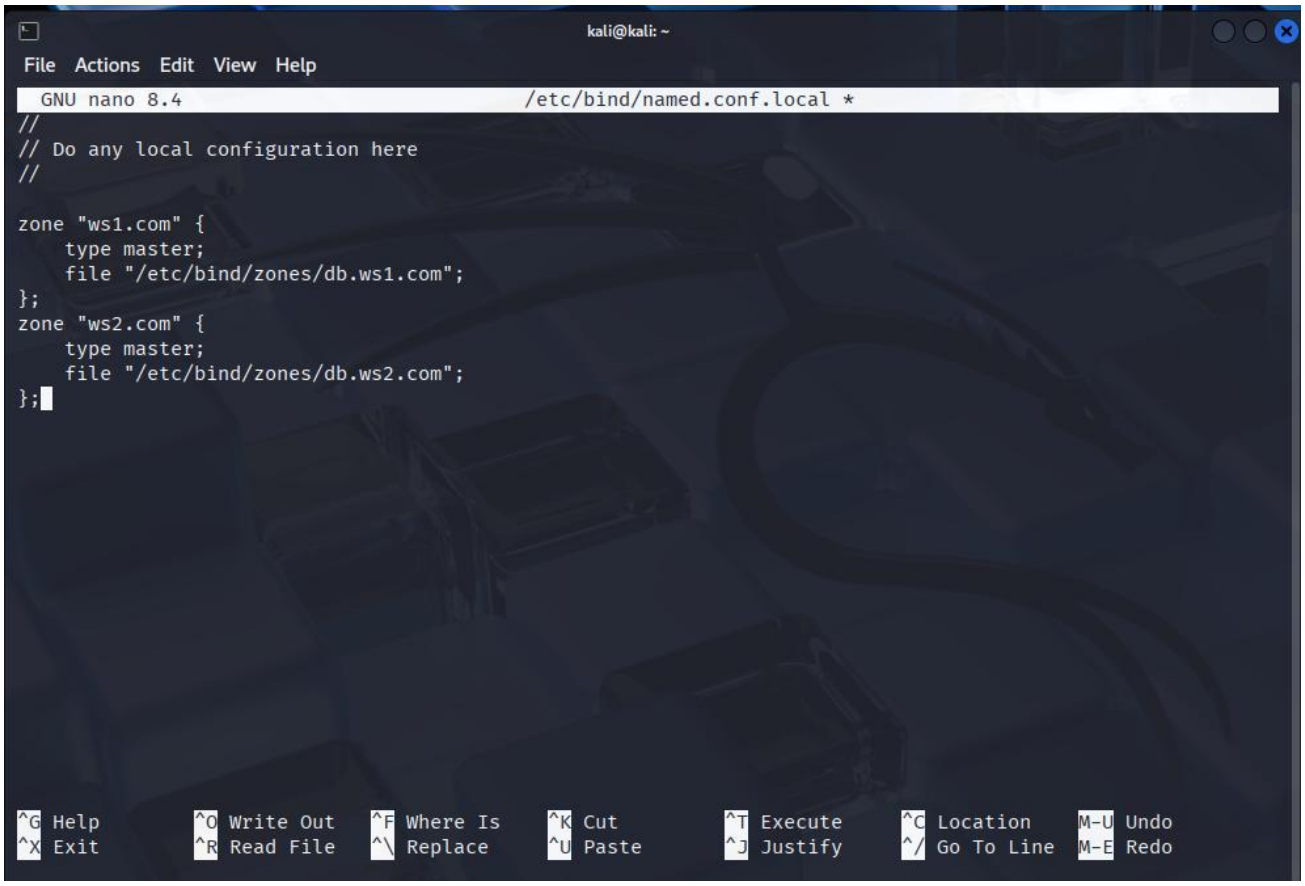

Figure 25: BIND9 Define Zone Configuration

- sudo nano /etc/bind/zones/db.ws1.com (configure zone file)

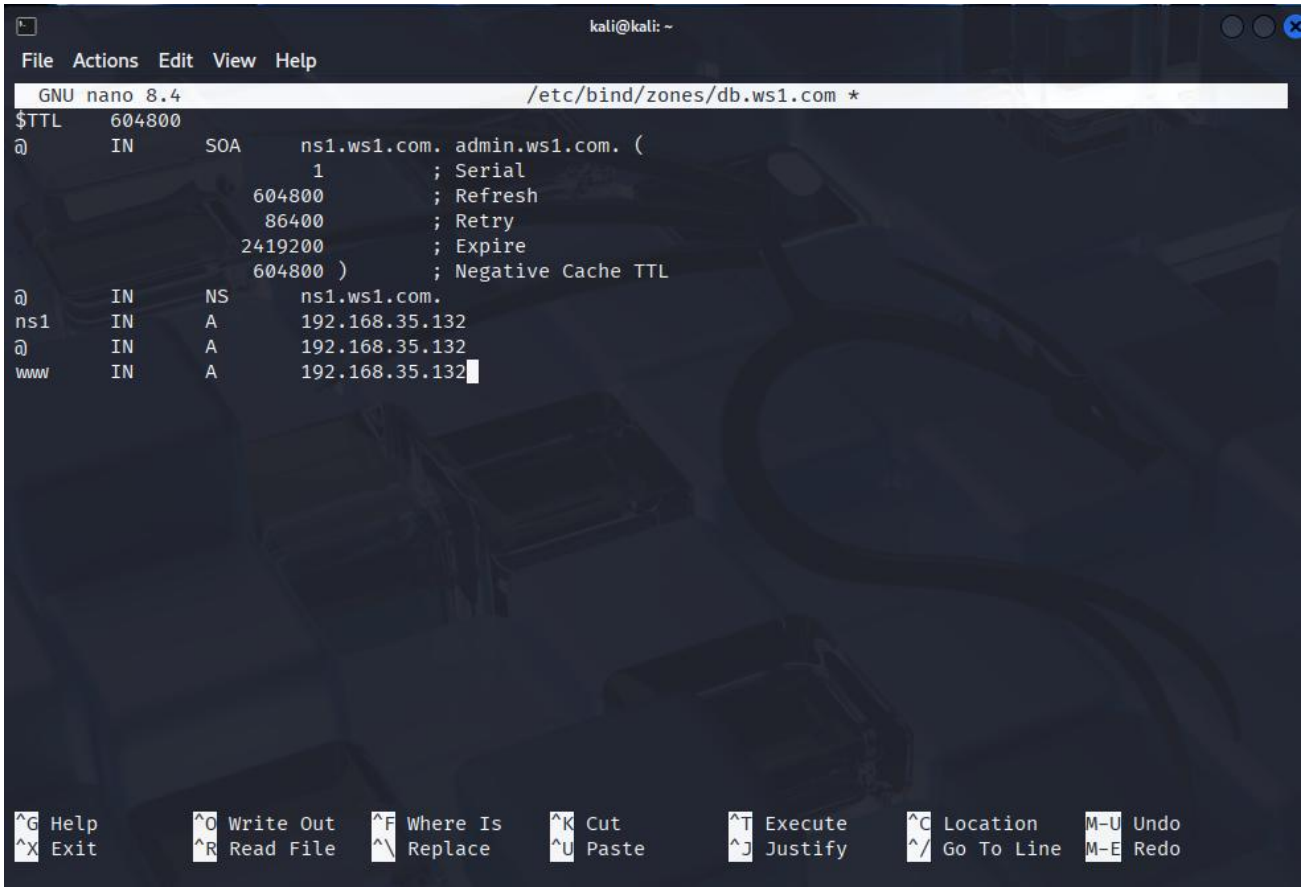

Figure 26: Zone file Configuration for ws1.com

- sudo nano /etc/bind/zones/db.ws2.com (configure zone file)

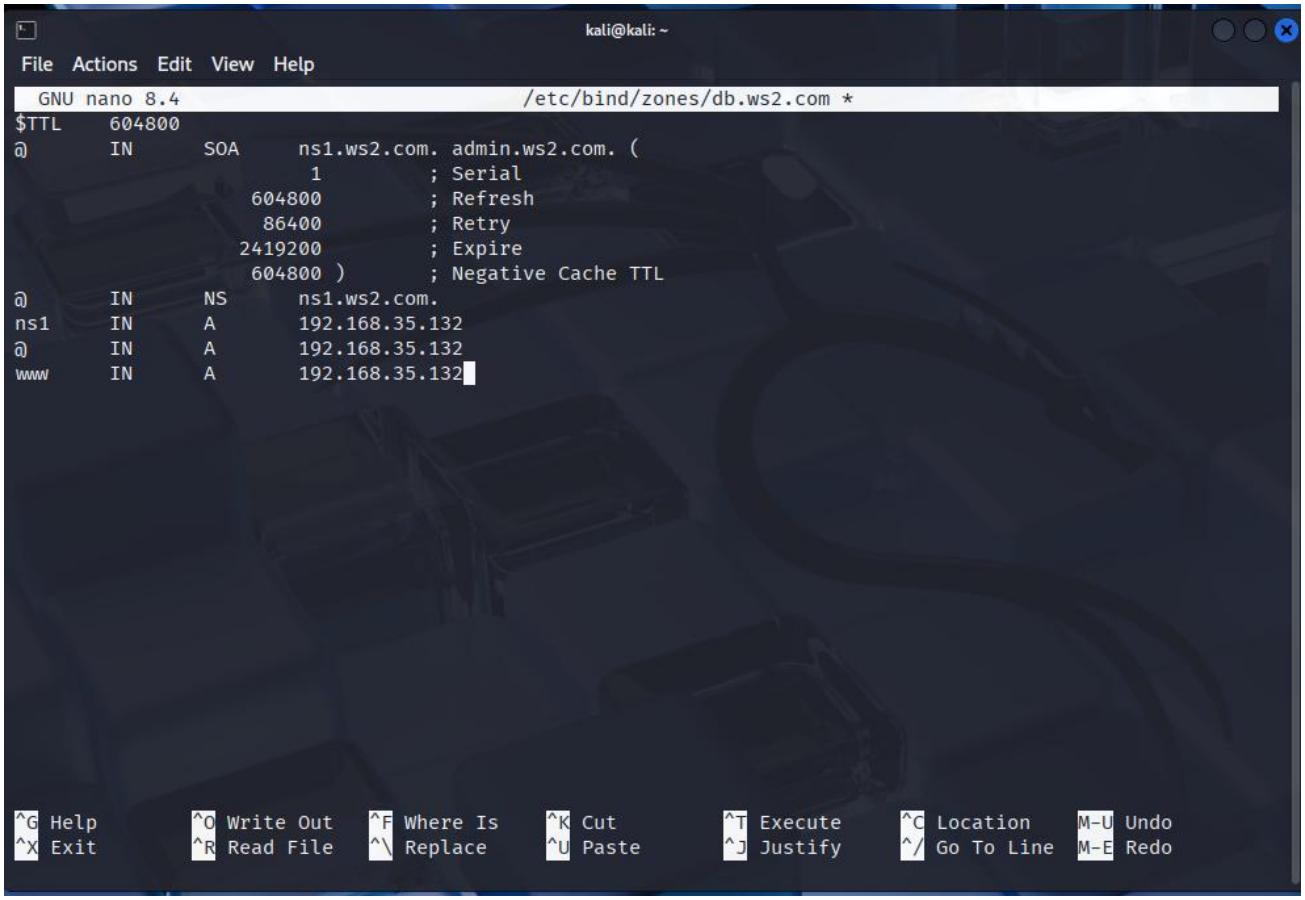

Figure 27: Zone file Configuration for ws2.com

- sudo chown -R bind:bind /etc/bind/zones
- sudo chmod -R 755 /etc/bind/zones
- sudo named-checkzone ws1.com /etc/bind/zones/db.ws1.com
- sudo named-checkzone ws2.com /etc/bind/zones/db.ws2.com
- sudo systemctl restart named

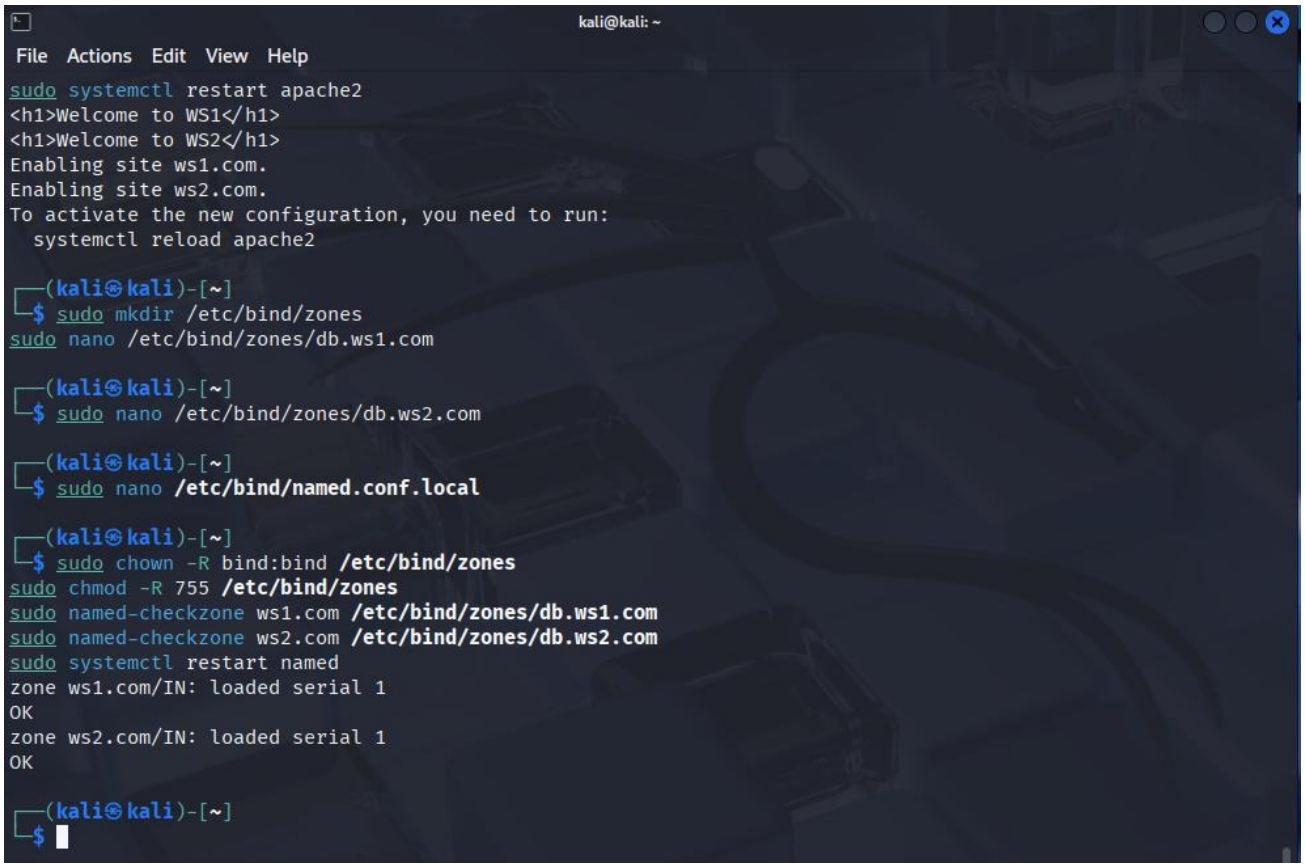

Figure 28: Status of BIND9

UFW Configuration: UFW is configured with **`sudo ufw allow 100/tcp`** and **`sudo ufw allow 53`** to permit HTTP and DNS traffic, respectively. The firewall is enabled with sudo ufw enable, and **`sudo ufw status`** verifies the rules. UFW ensures that only authorized traffic reaches Apache2 and BIND9, protecting client-server interactions.

UFW Configuration Commands:

- sudo ufw allow 100/tcp
- sudo ufw allow 53
- sudo ufw enable
- sudo ufw status

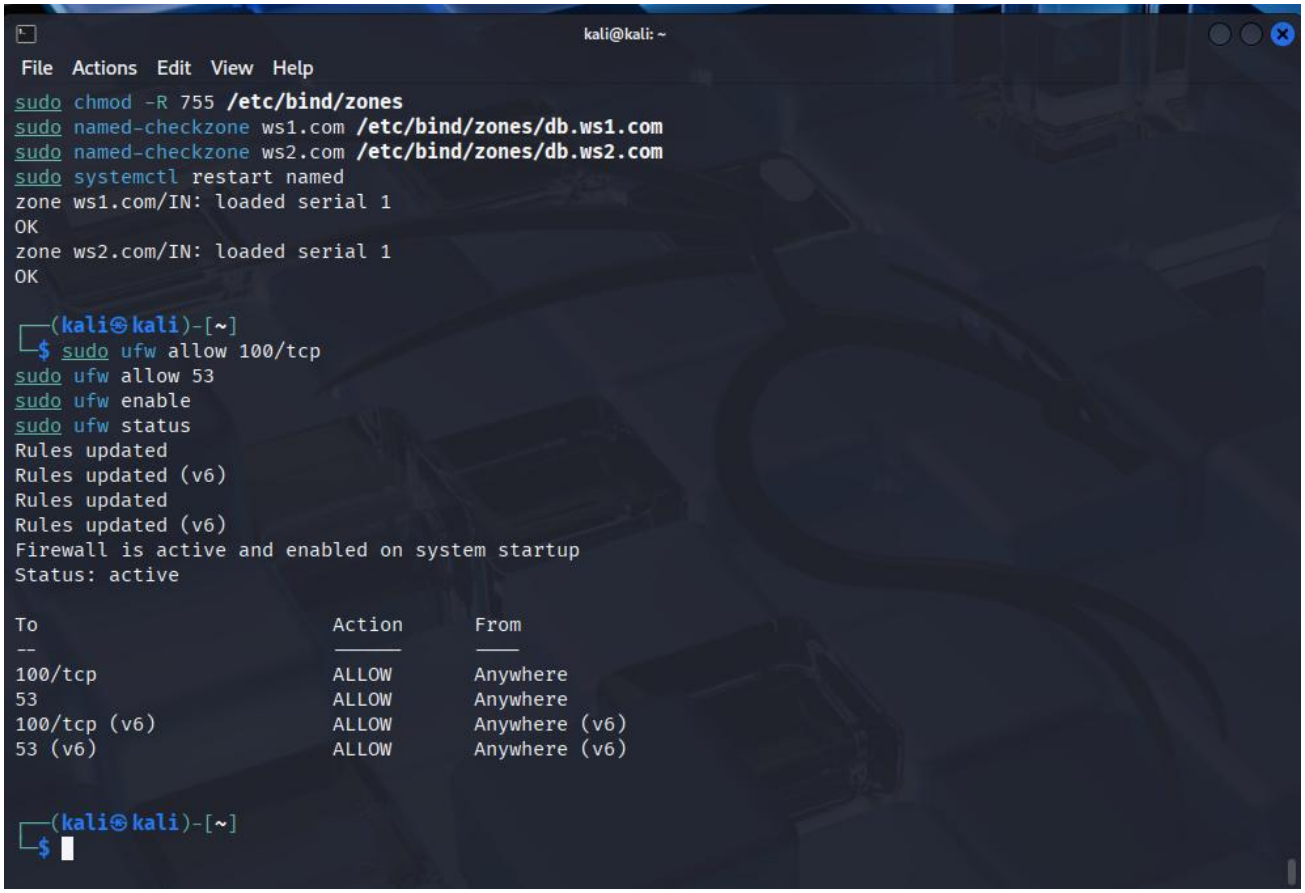

Figure 29: Firewall Configuration for ports allowed on Web and DNS Server

2. ADVANCE WEB DEPLOYMENT WITH DATABASE

Web Overview:

Aspect	Details
Domains	**`ws1.com`** and **`ws2.com`** (DVWA, dynamic), hosted on web/DNS server (**`192.168.35.132`**). Client sends DNS queries to BIND9, resolving to **`192.168.35.132`**, enabling Apache2 to serve dynamic content via MariaDB.
Servers	Web/DNS server (**`192.168.35.132`**) runs Apache2 (port 100) and BIND9 (port 53); Database server (**`192.168.35.133`**) runs MariaDB (port 3306). BIND9 resolves domains to Apache2, which queries MariaDB for dynamic data, secured by UFW.
Apache2	Hosts **`ws1.com`** and **`ws2.com`** at **`/var/www/ws1/public_html/DVWA`** and **`/var/www/ws2/public_html/DVWA`**, using PHP to query MariaDB

Aspect	Details
	(`192.168.35.133:3306`). Apache2 matches `ServerName` to BIND9's IPs, secured by UFW.
MariaDB	Hosts `dvwa_ws1` and `dvwa_ws2` databases on `192.168.35.133:3306` with user `huynhan`. Apache2's PHP scripts query MariaDB, retrieving dynamic data for DVWA, secured by UFW and authentication.
BIND9	Resolves `ws1.com` and `ws2.com` to `192.168.35.132` via zone files. Apache2's `ServerName` maps to IPs, enabling DVWA to query MariaDB, secured by UFW.
UFW	Allows ports 100 (HTTP) and 53 (DNS) on web/DNS server, and 3306 (MariaDB) on database server, protecting all services and allowing client and web-to-database traffic.

Network Configuration: On the Database server (`192.168.35.133`), `ip addr show eth0` confirms the IP, ensuring NAT network communication. The command cat `/etc/resolv.conf` verifies `nameserver 192.168.35.2`, and connectivity is tested with `ping -c 4 192.168.35.2` and `ping -c 4 8.8.8.8`. This setup enables MariaDB to receive queries from the web server and supports package installations.

Network Configuration Commands:

- ip addr show eth0

Figure 30: Default Database Settings

- cat /etc/resolv.conf
- ping -c 4 192.168.35.2
- ping -c 4 8.8.8.8
- sudo nano /etc/resolv.conf (add nameserver 8.8.8.8, nameserver 1.1.1.1)

MariaDB Configuration: To allow remote connections from **`192.168.35.132`**, the MariaDB configuration is updated with **`sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf`**, setting **`bind-address = 0.0.0.0`**. Databases are created with **`sudo mariadb -u root -p`**:

```
CREATE DATABASE dvwa_ws1;
CREATE DATABASE dvwa_ws2;
CREATE USER 'huynhan'@'192.168.35.132' IDENTIFIED BY 'huynhan1812';
GRANT ALL PRIVILEGES ON dvwa_ws1.* TO 'huynhan'@'192.168.35.132';
GRANT ALL PRIVILEGES ON dvwa_ws2.* TO 'huynhan'@'192.168.35.132';
FLUSH PRIVILEGES;
EXIT;
```

MariaDB is restarted with **`sudo systemctl restart mariadb`**, and **`mariadb -u root -p -e "SHOW DATABASES;"`** confirms the databases. MariaDB interacts with Apache2's PHP scripts to provide dynamic data for DVWA on both websites.

MariaDB Configuration Commands:

- sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf (set bind-address = 0.0.0.0)

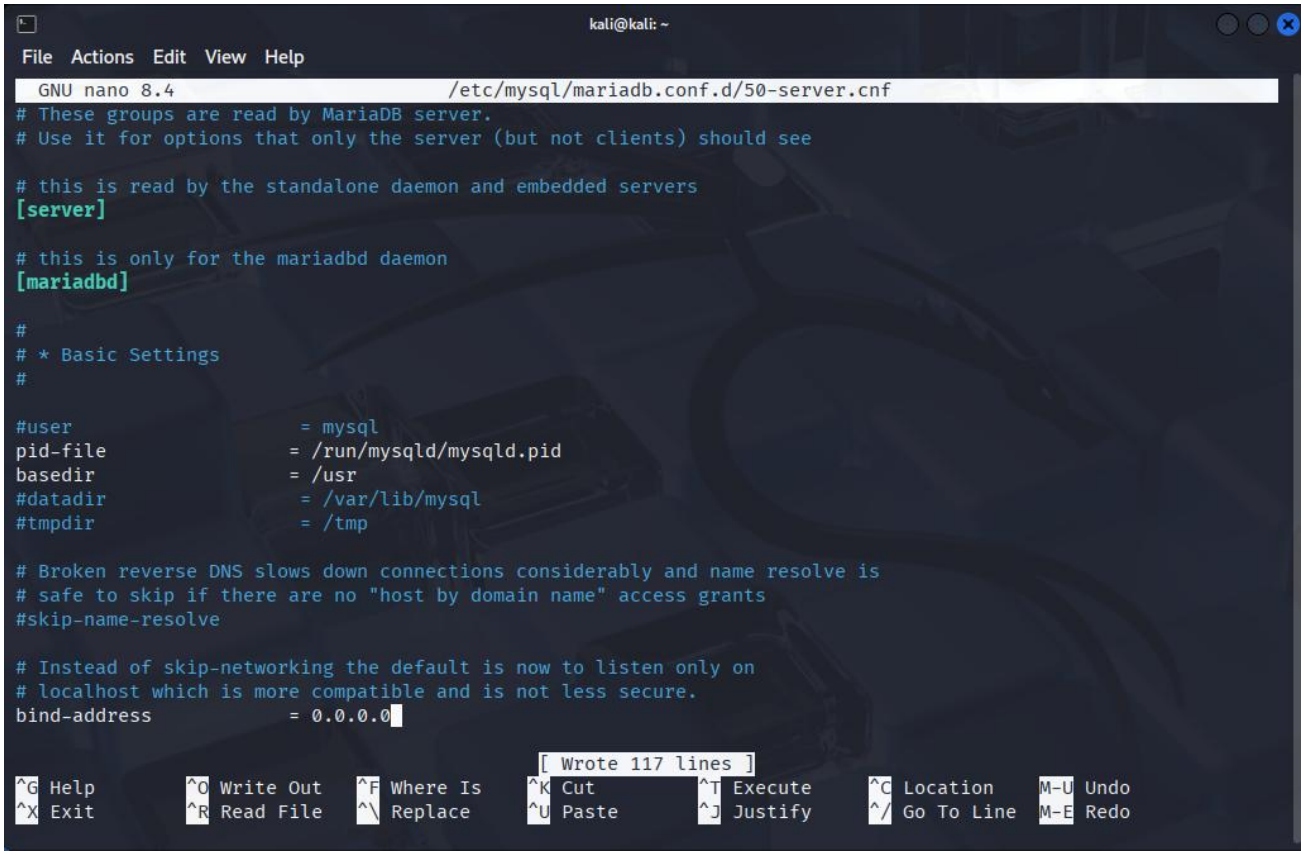

Figure 31: Bind Address Configuration of MaridaDB

- sudo mariadb -u root -p
- CREATE DATABASE dvwa_ws1;
- CREATE DATABASE dvwa_ws2;
- CREATE USER 'huynhan'@'192.168.35.132' IDENTIFIED BY 'huynhan1812';
- GRANT ALL PRIVILEGES ON dvwa_ws1.* TO 'huynhan'@'192.168.35.132';
- GRANT ALL PRIVILEGES ON dvwa_ws2.* TO 'huynhan'@'192.168.35.132';
- FLUSH PRIVILEGES;
- EXIT;
- sudo systemctl restart mariadb
- mariadb -u root -p -e "SHOW DATABASES;"

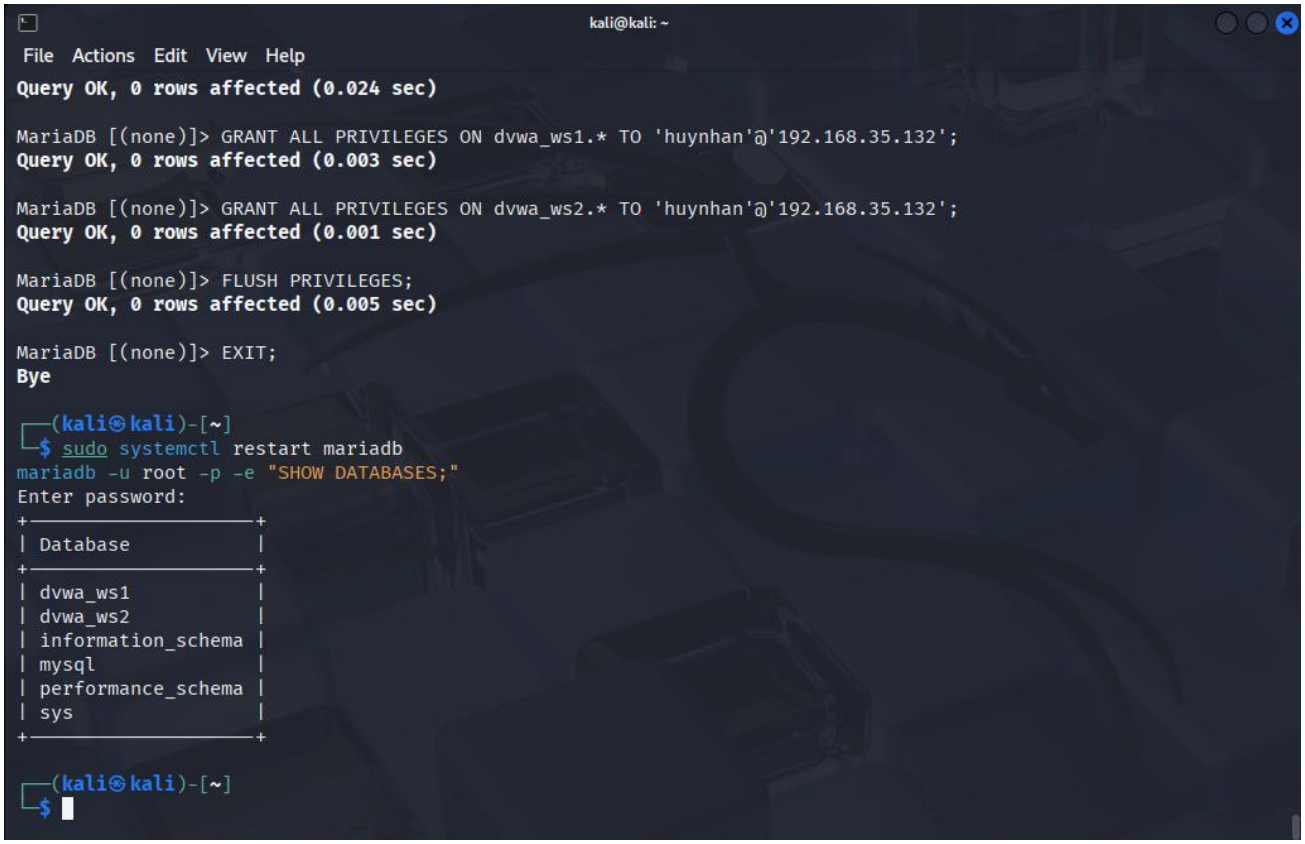

Figure 32: DVWA Database Status in MariaDB

UFW Configuration: UFW is configured with **`sudo apt install ufw -y`**, **`sudo ufw allow 3306/tcp`**, **`sudo ufw enable`**, and **`sudo ufw status`** to allow MariaDB traffic while securing the server. This ensures safe database interactions with the web server.

UFW Configuration Commands:

- sudo apt install ufw -y
- sudo ufw allow 3306/tcp
- sudo ufw enable
- sudo ufw status

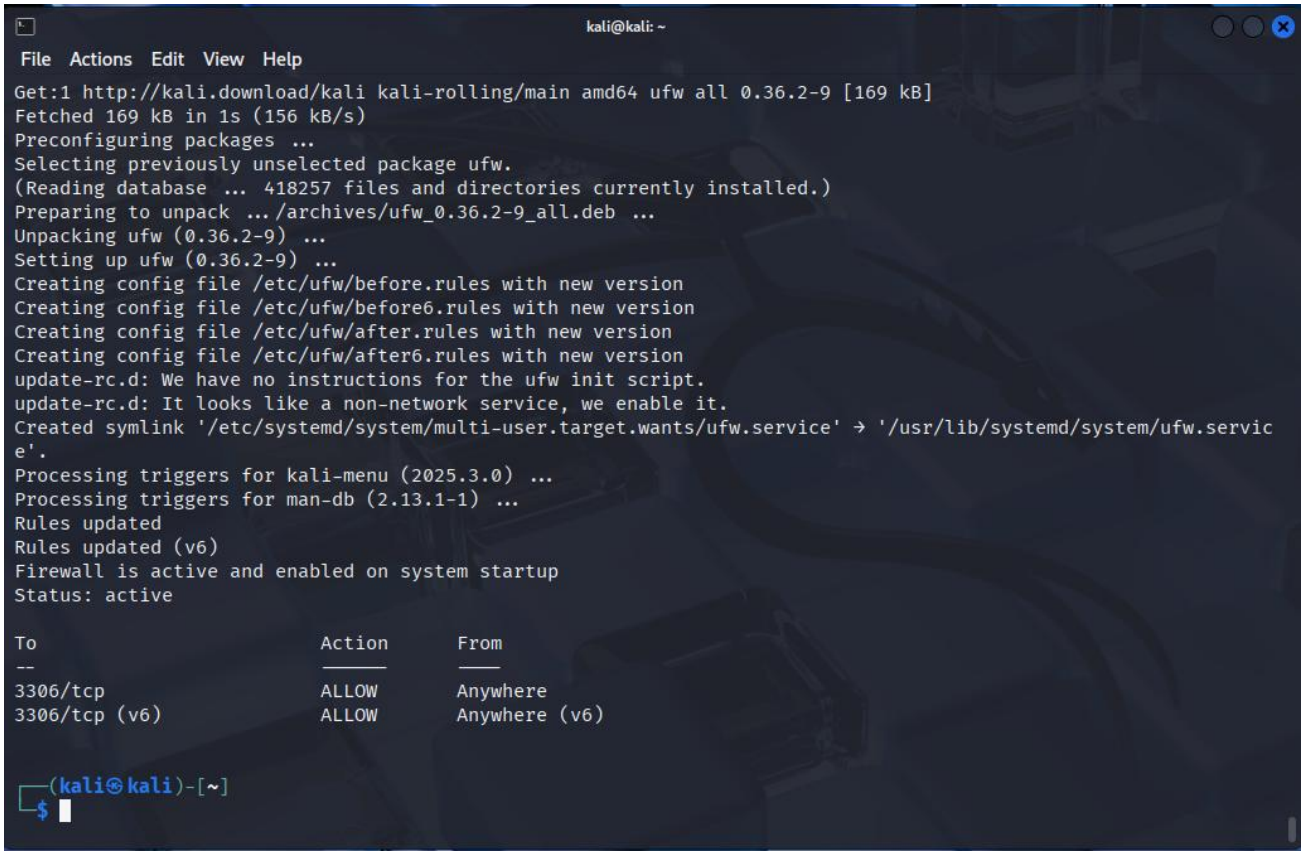

Figure 33: Firewall Configuration for ports allowed on Database Server

DVWA Configuration: DVWA configurations are updated to connect to the respective databases. For **`ws1.com`**, **`sudo nano /var/www/ws1/public_html/DVWA/config/config.inc.php`** sets:

```
$_DVWA['db_server'] = '192.168.35.133';
$_DVWA['db_database'] = 'dvwa_ws1';
$_DVWA['db_user'] = 'huynhan';
$_DVWA['db_password'] = 'huynhan1812';
```

For **`ws2.com`**, **`sudo nano /var/www/ws2/public_html/DVWA/config/config.inc.php`** sets:

```
$_DVWA['db_server'] = '192.168.35.133';
$_DVWA['db_database'] = 'dvwa_ws2';
$_DVWA['db_user'] = 'huynhan';
$_DVWA['db_password'] = 'huynhan1812';
```

Connectivity is tested with **`mariadb -u huynhan -p -h 192.168.35.133 -e "SHOW DATABASES;"`** to ensure Apache2 can access MariaDB. This configuration links DVWA to its database, enabling dynamic content generation.

DVWA Configuration Commands:

- cd /var/www/ws1/public_html/DVWA/config
- sudo cp config.inc.php.dist config.inc.php
- sudo nano /var/www/ws1/public_html/DVWA/config/config.inc.php (set database parameters)

```
kali@kali: ~
File  Actions  Edit  View  Help
  GNU nano 8.4                    config.inc.php *
<?php

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
#   Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ?: 'MySQL';
#$DBMS = 'PGSQL'; // Currently disabled

# Database variables
#   WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
#   Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
#   See README.md for more information on this.
$_DVWA = array();
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '192.168.35.133';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa_ws1';
$_DVWA[ 'db_user' ]     = getenv('DB_USER') ?: 'huynhan';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: 'huynhan1812';
$_DVWA[ 'db_port']      = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
#   Used for the 'Insecure CAPTCHA' module
#   You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA[ 'recaptcha_public_key' ]  = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
$_DVWA[ 'recaptcha_private_key' ] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo
```

Figure 34: DVWA Database Configuration for ws1.com

- cd /var/www/ws2/public_html/DVWA/config
- sudo cp config.inc.php.dist config.inc.php
- sudo nano /var/www/ws2/public_html/DVWA/config/config.inc.php (set database parameters)

```
kali@kali: /var/www/ws1/public_html/DVWA/config
File  Actions  Edit  View  Help
  GNU nano 8.4                          config.inc.php
<?php

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
#   Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ?: 'MySQL';
#$DBMS = 'PGSQL'; // Currently disabled

# Database variables
#   WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
#   Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
#   See README.md for more information on this.
$_DVWA = array();
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '192.168.35.133';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa_ws2';
$_DVWA[ 'db_user' ]     = getenv('DB_USER') ?: 'huynhan';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: 'huynhan1812';
$_DVWA[ 'db_port']      = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
#   Used for the 'Insecure CAPTCHA' module
#   You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA[ 'recaptcha_public_key' ]  = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
$_DVWA[ 'recaptcha_private_key' ] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

^G Help      ^O Write Out  ^F Where Is   ^K Cut      ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste    ^J Justify    ^/ Go To Line M-E Redo
```

Figure 35: DVWA Database Configuration for ws2.com

- mariadb -u huynhan -p -h 192.168.35.133 -e "SHOW DATABASES;"

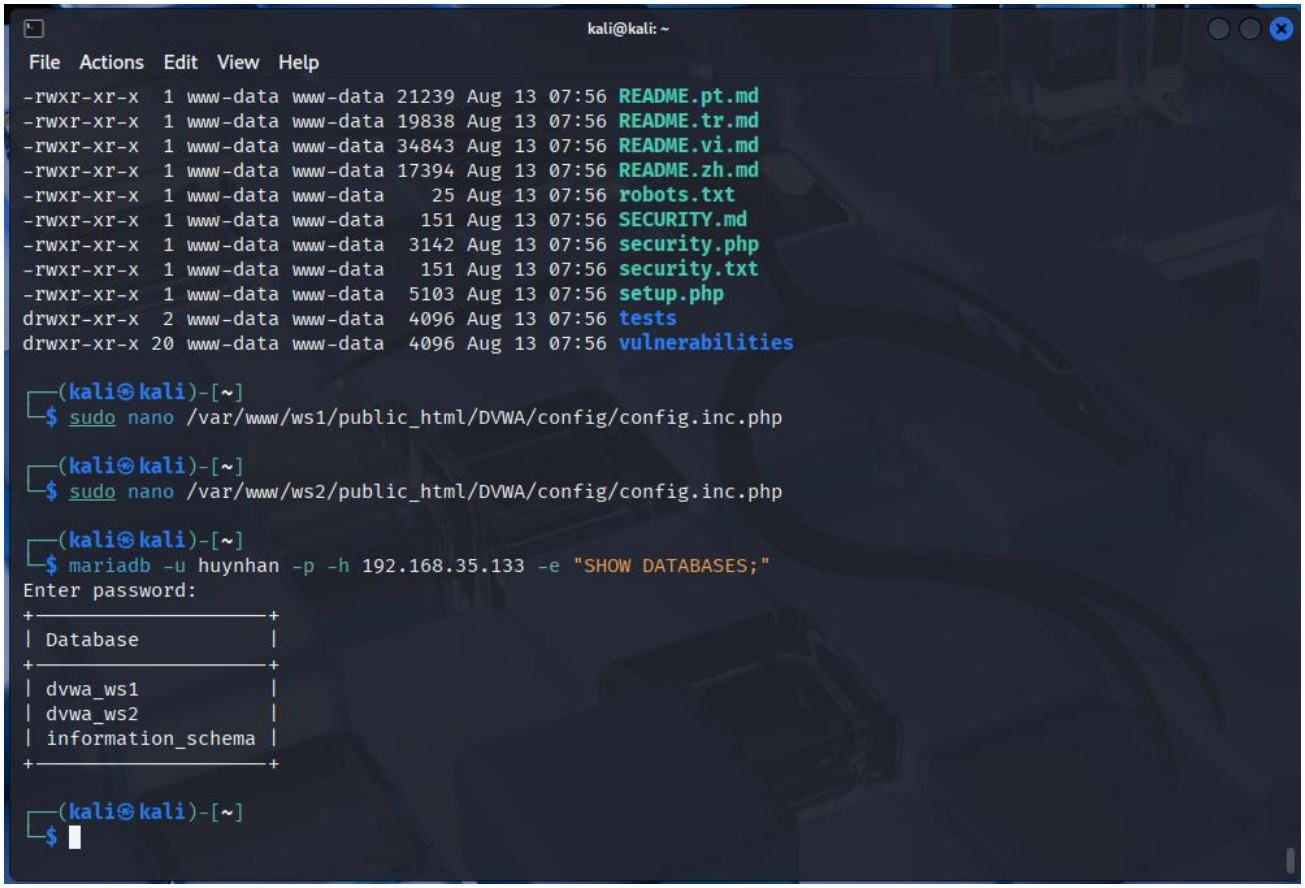

Figure 36:MariaDB Connectivity on Database Server

Apache2 Configuration: The virtual host for **`ws1.com`** is updated with **`sudo nano /etc/apache2/sites-available/ws1.com.con`**f:

```
<VirtualHost *:100>
    ServerAdmin admin@ws1.com
    ServerName ws1.com
    ServerAlias www.ws1.com
    DocumentRoot /var/www/ws1/public_html
    ErrorLog ${APACHE_LOG_DIR}/ws1-error.log
    CustomLog ${APACHE_LOG_DIR}/ws1-access.log combined
    <Directory /var/www/ws1/public_html/DVWA>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    <FilesMatch "\.php$">
        SetHandler application/x-httpd-php
    </FilesMatch>
    <FilesMatch "\.phps$">
        SetHandler application/x-httpd-php-source
    </FilesMatch>
</VirtualHost>
```

The same is done for **`ws2.com`**. PHP is enabled with **`sudo a2enmod php8.4`**, and **`mod_rewrite`** is enabled with **`sudo a2enmod rewrite`** to support DVWA's features like CAPTCHA. Apache2 is restarted with **`sudo systemctl restart apache2`**. Apache2 processes PHP scripts, querying MariaDB for dynamic content.

Apache2 Configuration Commands:

- sudo nano /etc/apache2/sites-available/ws1.com.conf (configure virtual host with PHP handler)

Figure 37: Configuration virtual host for ws1.com by DVWA on Apache

- sudo nano /etc/apache2/sites-available/ws2.com.conf (configure virtual host with PHP handler)

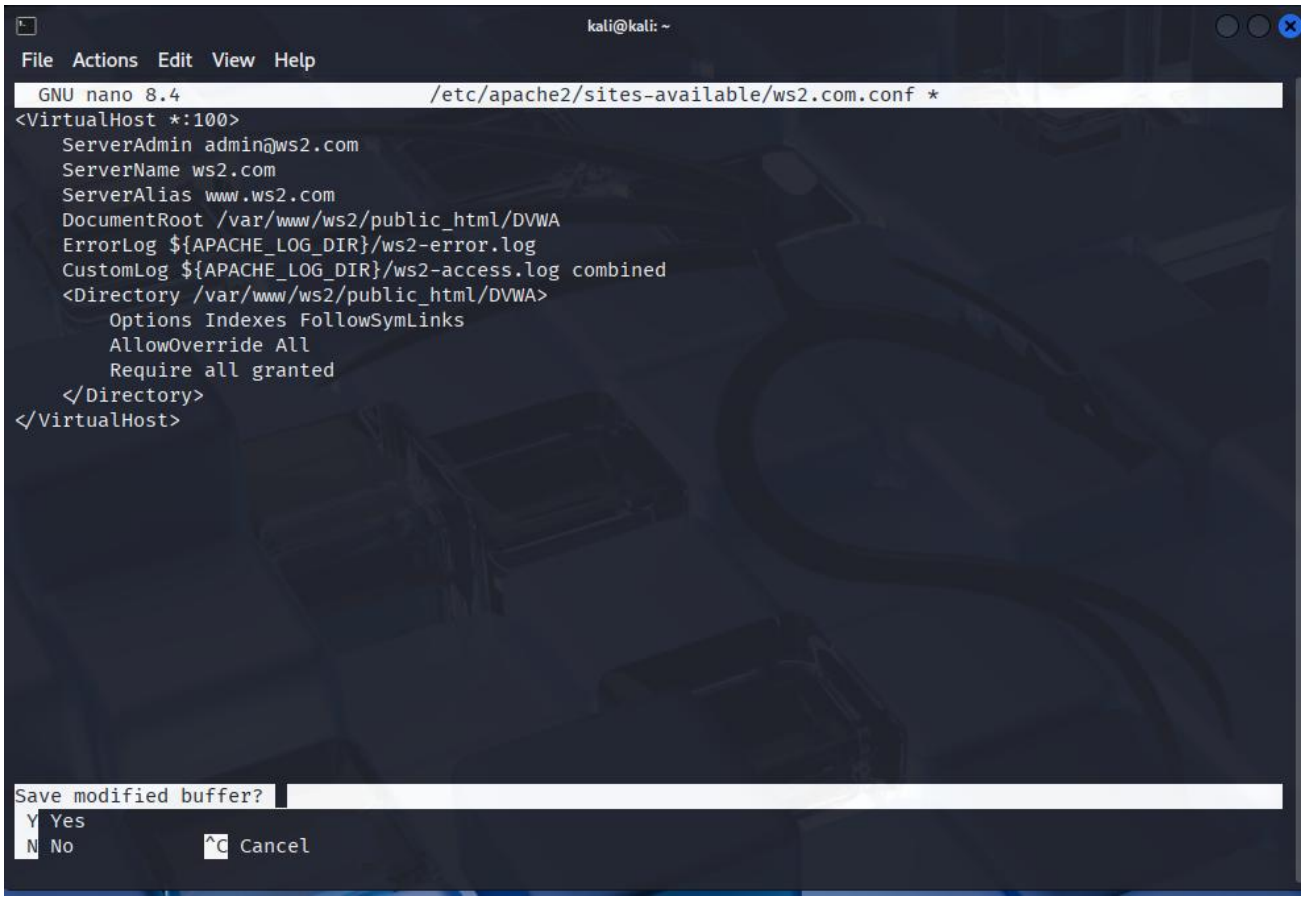

Figure 38: Configuration virtual host for ws2.com by DVWA on Apache

- sudo a2enmod php8.4
- sudo a2enmod rewrite
- sudo systemctl restart apache2

III. RESULTS

1. BASIC WEB DEPLOYMENT

The basic setup successfully meets the requirement to install a web server (Apache2) and DNS server (BIND9) on a single Kali Linux VM (192.168.35.132) and deploy two websites (**`ws1.com`** and **`ws2.com`**) serving static content. Apache2 was installed and configured to listen on port 100, hosting virtual hosts for **`ws1.com`** and **`ws2.com`** at **`/var/www/ws1/public_html`** and **`/var/www/ws2/public_html`**, respectively, with **`index.html`** files displaying "Welcome to WS1!" and "Welcome to WS2!". BIND9 was installed and configured to resolve both domains to 192.168.35.132 using zone files **`/etc/bind/zones/db.ws1.com`** and **`db.ws2.com`**. The system was verified through the following steps: network connectivity was confirmed with **`ping 192.168.35.2`** (gateway) and **`ping 8.8.8.8`** (public internet); DNS resolution was tested with **`nslookup ws1.com 192.168.35.132`** and **`nslookup ws2.com 192.168.35.132`**, confirming both domains resolve to 192.168.35.132; and web access was validated by accessing **`http://ws1.com:100`** and **`http://ws2.com:100`** in a browser, displaying the static content as expected. UFW was configured to allow ports 53 (DNS) and 100 (HTTP), ensuring secure access.

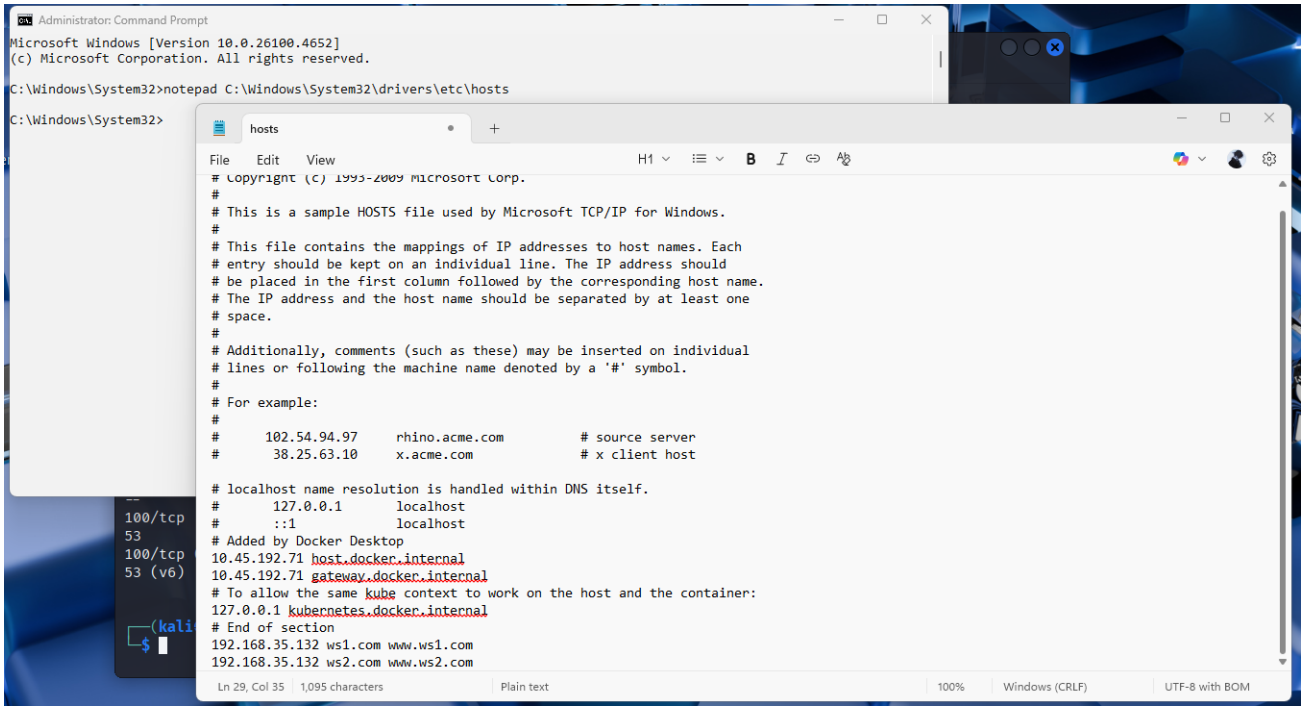

Figure 39: DNS Resolution for Window Client

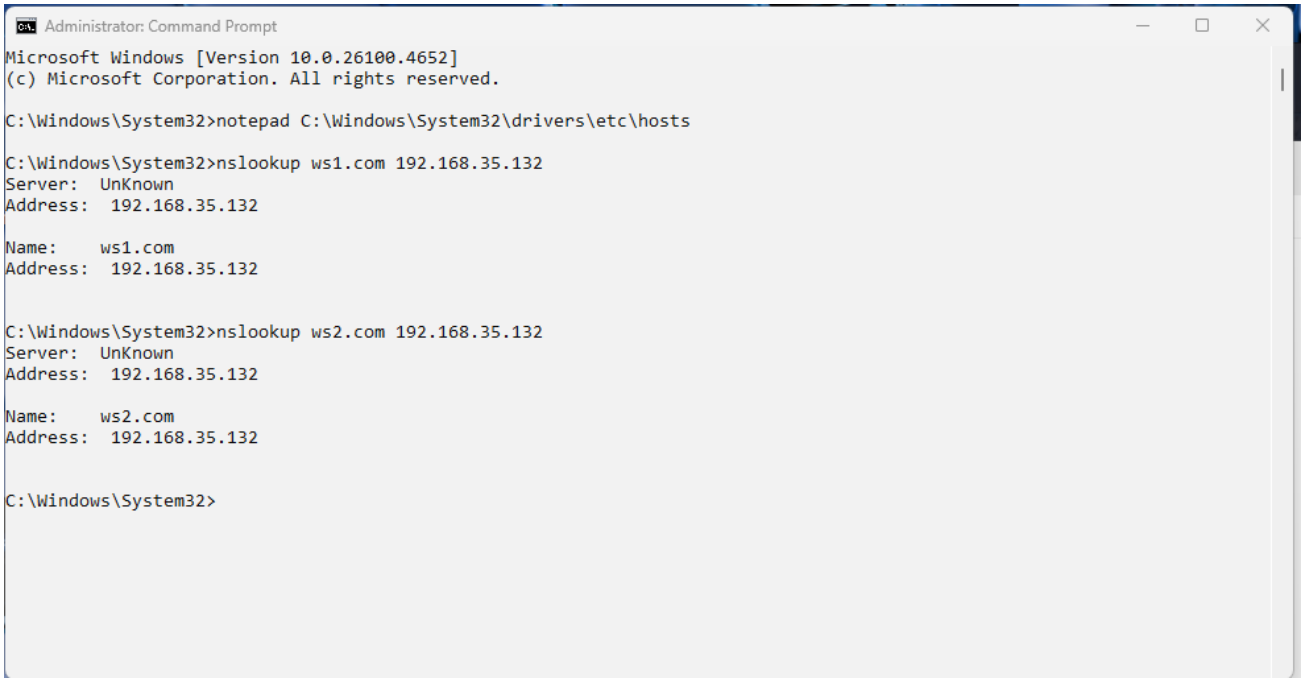

Figure 40: Nslookup for ws1.com and ws2.com

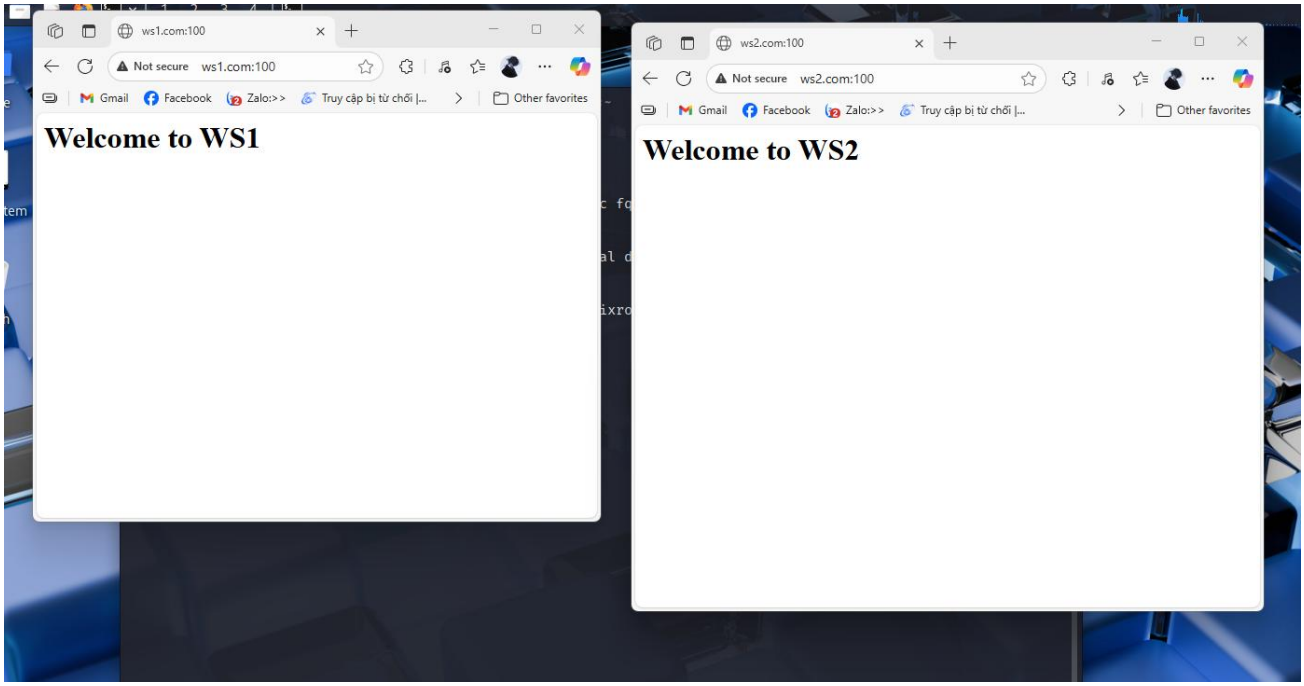

Figure 41: Static Content Result

2. ADVANCE WEB DEPLOYMENT WITH DATABASE

The advanced setup successfully meets the requirement to install a web server (Apache2) and DNS server (BIND9) on one Kali Linux VM (192.168.35.132), deploy two websites (**`ws1.com`** and **`ws2.com`**) with dynamic content using DVWA, and configure both websites to connect to separate databases (**`dvwa_ws1`** and **`dvwa_ws2`**) on a different server running MariaDB (192.168.35.133). Apache2 was installed and configured on the Web/DNS server to host **`ws1.com`** and **`ws2.com`** at **`/var/www/ws1/public_html/DVWA`** and **`/var/www/ws2/public_html/DVWA`**, with PHP enabled to process dynamic content. BIND9 was installed and configured to resolve both domains to

192.168.35.132. MariaDB was installed on the database server, hosting **dvwa_ws1** and **dvwa_ws2**, with user **huynhan** granted access from 192.168.35.132. The DVWA configurations (**config.inc.php**) were set to connect **ws1.com** to **dvwa_ws1** and **ws2.com** to **dvwa_ws2**, ensuring isolated data access. Verification included: network connectivity on the database server with **ip addr show eth0** and **ping -c 4 192.168.35.2**; database access with **mariadb -u huynhan -p -h 192.168.35.133 -e "SHOW DATABASES;"** from the Web/DNS server, confirming access to **dvwa_ws1** and **dvwa_ws2**; DNS resolution with **nslookup ws1.com 192.168.35.132** and **nslookup ws2.com 192.168.35.132**; and web access by navigating to **http://ws1.com:100/DVWA/setup.php** and **http://ws2.com:100/DVWA/setup.php** to initialize databases, then to **http://ws1.com:100/DVWA/login.php** and **http://ws2.com:100/DVWA/login.php** with credentials **admin/password**, displaying the DVWA login page. The DVWA security level was set to 'low' for testing, and screenshots confirmed functionality. UFW was configured to allow ports 53 and 100 on the Web/DNS server and port 3306 on the database server, ensuring secure communication.

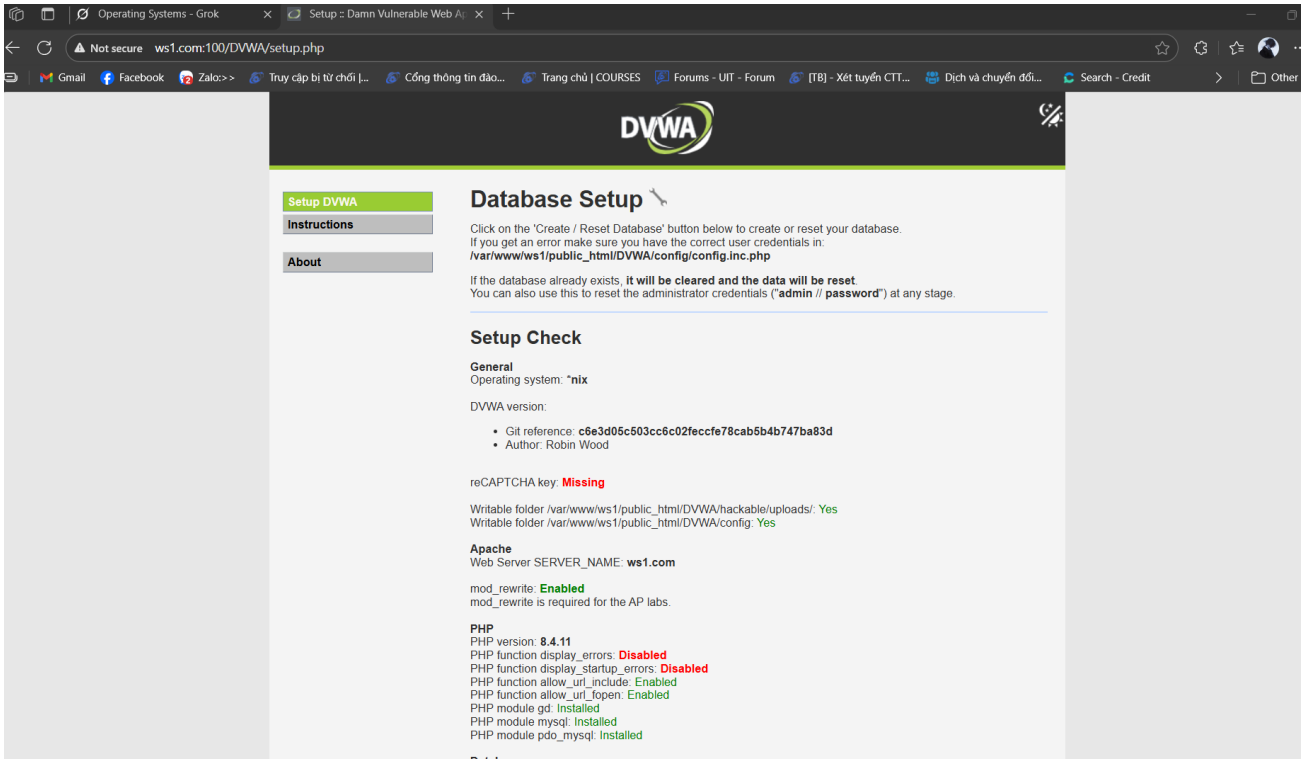

Figure 42: DVWA Setup Page

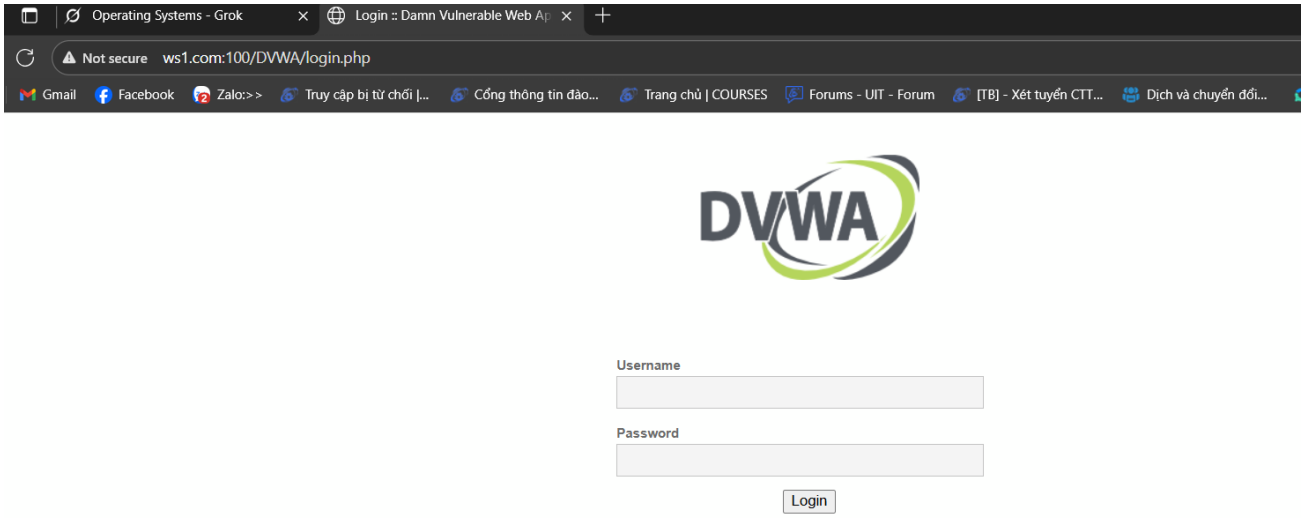

Figure 43: DVWA Login Page

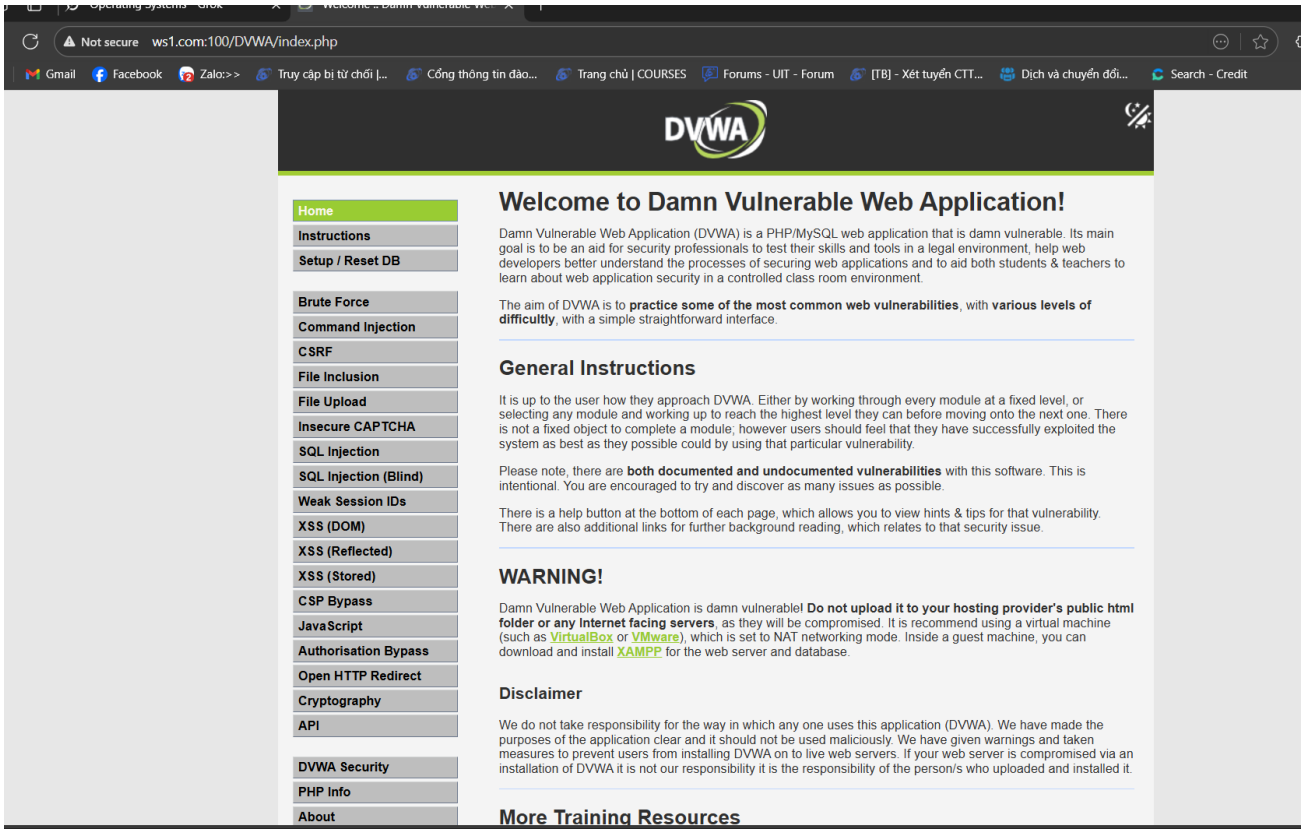

Figure 44: DVWA Web Application

IV. REFERENCES

- Apache Software Foundation. (n.d.). *Apache HTTP Server Documentation*. Retrieved from https://httpd.apache.org/docs/2.4/
- Internet Systems Consortium. (n.d.). *BIND 9 Administrator Reference Manual*. Retrieved from https://bind9.readthedocs.io/en/latest/
- MariaDB Foundation. (n.d.). *MariaDB Documentation*. Retrieved from https://mariadb.com/kb/en/documentation/
- Digininja. (n.d.). *Damn Vulnerable Web Application (DVWA) GitHub Repository for Cloning*. Retrieved from https://github.com/digininja/DVWA (Used for git clone https://github.com/digininja/DVWA.git)
- Ubuntu Community. (n.d.). *UFW - Uncomplicated Firewall*. Retrieved from https://help.ubuntu.com/community/UFW
- PHP Documentation Group. (n.d.). *PHP Manual*. Retrieved from https://www.php.net/manual/en/
- Kali Linux Documentation. (2025). *Kali Linux Official Documentation*. Retrieved from https://www.kali.org/docs/
- Debian Wiki. (n.d.). *Configuring Apache2 Virtual Hosts*. Retrieved from https://wiki.debian.org/Apache
- Debian Wiki. (n.d.). *BIND9 Configuration Guide*. Retrieved from https://wiki.debian.org/Bind9